



ORACLE
NetSuite

Shipping Partner Plug-in - BETA

2019.1

September 3, 2025



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

Shipping Partner Plug-in Overview	1
Shipping Partner Plug-in Features in NetSuite	3
Developing a Shipping Partner Plug-in Implementation	9
Step 1. Enable Features for Development	10
Step 2. Customize Shipping Partner Entry Forms	11
Step 3. Create Custom Record Type	12
Step 4. Create the Plug-in Implementation Script File	12
Step 5. Add and Activate the Plug-in Implementation	16
Step 6. Bundle the Plug-in Implementation	18
Administering a Shipping Partner Plug-in Implementation	18
Enable Features for Shipping Partner Plug-in	19
Install a Shipping Partner Plug-in Bundle	19
Activate the Plug-in	19
Register Shipping Partner	20
Create Shipping Items	20
Shipping Partner Interface Definition	23
getServiceGroups(GetServiceGroupsInput input, GetServiceGroupsOutput output)	24
GetServiceGroupsInput	25
GetServiceGroupsOutput	25
displayRegistration(DisplayRegistrationInput input, DisplayRegistrationOutput output)	46
DisplayRegistrationInput	47
DisplayRegistrationOutput	48
processRegistration(ProcessRegistrationInput input, ProcessRegistrationOutput output)	50
ProcessRegistrationInput	50
ProcessRegistrationOutput	52
calculateRates(CalculateRatesInput input, CalculateRatesOutput output)	53
CalculateRatesInput	53
CalculateRatesOutput	75
processShipment(ProcessShipmentInput input, ProcessShipmentOutput output)	82
ProcessShipmentInput	83
ProcessShipmentOutput	84
voidShipment(VoidShipmentInput input, VoidShipmentOutput output)	100
VoidShipmentInput	100
VoidShipmentOutput	101
Input and Output Object Methods	101
getObjectFactory()	102
getNotificationsContainer()	107
SuiteScript Support	111
Exposed Shipping Partner Records and Subrecords	111
Shipping Partner Package	111
Shipping Partner Registration	112
Shipping Partner Shipment	115

Shipping Partner Plug-in Overview



Important: This document describes a beta feature. The contents of this feature are preliminary and may be changed or discontinued without prior notice. Any changes may impact the feature's operation with the NetSuite application. NetSuite warranties and product service levels shall not apply to the feature or the impact of the feature on other portions of the NetSuite application. NetSuite may review and monitor the performance and use of this feature. Note that this document is also considered to be beta and its contents are subject to change.

Use the Shipping Partner plug-in to integrate NetSuite with third-party shipping partners, similar to USPS and FedEx. You can define a plug-in implementation that integrates sales order and item fulfillment records with an external third-party shipping system to calculate shipping rates and provide shipping documents.

An implementation of the Shipping Partner plug-in defines the following functionality and features for a shipping partner in NetSuite:

- **Rate calculation.** Define rates based on the item details for a sales order or item details and packages for an item fulfillment. NetSuite provides details about a sales order or item fulfillment transaction to the plug-in implementation, including item name, quantity, unit price and weight, and associated package details.
- **Service groups and related services.** A service group is a grouping of related shipping services. Each plug-in implementation defines the service groups and the services for the groups. For example, a shipping partner may want a group of services specifically for processing letters and another for processing boxes. Use the service groups and services to create shipping items for the shipping partner.
- **Partner registration.** Defines the process for registering a shipping partner with NetSuite and storing the credentials used to authenticate shipping services with an external shipping partner account.
- **Shipping documents.** Documents in PDF format that are associated with a item fulfillment, including shipping labels, return labels, and other related printed collateral.
- **Custom forms.** Forms include registration forms and shipment and package details. The registration form is used when you register a new shipping partner in NetSuite. The shipment and package details are available on item fulfillment record forms when a user selects a shipping method for the shipping partner.

To implement a shipping partner with the Shipping Partner plug-in, you must identify the service groups and related services and identify the third-party external shipping account that you want to use to calculate real-time shipping rates. Then, a developer creates an implementation of the Shipping Partner plug-in. The implementation can communicate with the external shipping account to calculate shipping rates and provide shipping documents.

Any plug-in implementation appears as a shipping partner when you create shipping items for the NetSuite account. You create a shipping item based on the Shipping Partner plug-in implementation, and then specify the shipping item as the shipping method for a sales order or item fulfillment. After you select the shipping method, you can calculate shipping rates, based on the item properties in the transaction.

After a developer defines and tests the plug-in implementation, you use SuiteBundler to distribute the plug-in implementation as a SuiteApp to other NetSuite accounts.

For information about the Shipping Partner plug-in, see the following topics:

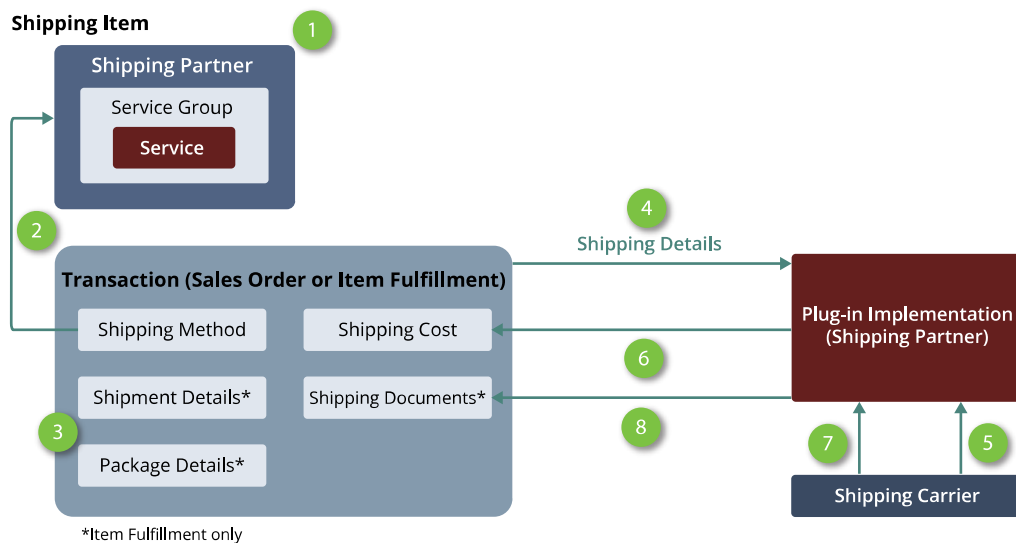
NetSuite Role	For more information, see ...
All roles	Shipping Partner Plug-in Features in NetSuite

NetSuite Role	For more information, see ...
Developer	Developing a Shipping Partner Plug-in Implementation
Administrator	Administering a Shipping Partner Plug-in Implementation

Important: The Shipping Partner plug-in does not currently support SuiteScript 2.0. Only SuiteScript 1.0 is supported.

Shipping Partner Plug-In Implementation Process Flow

The following figure describes the process for calculating shipping rates and providing shipping documents with a Shipping Partner plug-in implementation:



- 1. Create the shipping item.** An administrator creates a shipping item based on the plug-in implementation. The shipping item includes the shipping partner, service group, and service defined by the plug-in implementation. Optionally, the administrator registers the shipping partner to enable calculation of real-time rates.
- 2. Select the shipping method for the transaction.** On a sales order or item fulfillment, a user selects the shipping partner as the carrier and selects the shipping method defined by the shipping item.
- 3. (Optional) Specify shipment and package details.** A user can specify shipment and package details on an item fulfillment record.
- 4. Calculate the shipping rate.** A user clicks the **Calculate** button for the shipping cost. NetSuite calculates rates using the logic defined by the plug-in implementation. Depending on the functionality defined by the plug-in, the plug-in implementation can send a request to the external shipping account to calculate the rates.
- 5. External system sends response.** The external shipping partner account sends the shipping cost details back to NetSuite.

6. **NetSuite displays the shipping cost.** NetSuite displays the shipping cost on the **Shipping** subtab and in the Summary panel for the transaction. The shipping cost is saved to the database when the user saves the transaction record.
7. **User prints shipping documents.** Depending on the setup of the shipping item and the plug-in implementation functionality, users can print shipping and return labels and other documents related to the transaction. The plug-in implementation connects to the external shipping partner account to generate the shipping documents.

Note: You must request shipping rates directly from the transaction record by clicking **Calculate** on the **Shipping** subtab. If shipping rates change after you save the transaction, you must open the transaction and recalculate shipping rates. The plug-in implementation does not independently request shipping rates from the external shipping partner account.

Shipping Partner Plug-in Features in NetSuite

To use the Shipping Partner plug-in in a NetSuite account, an administrator must install a bundle that contains an implementation of the Shipping Partner plug-in and enable the implementation.

For more information, see [Developing a Shipping Partner Plug-in Implementation](#) and [Administering a Shipping Partner Plug-in Implementation](#).

After an administrator installs a Shipping Partner bundle and enables a Shipping Partner plug-in implementation that represents a shipping carrier, you can complete the following tasks:

- [Create Shipping Items](#)
- [Register a Shipping Partner](#)
- [Calculate Shipping Rates](#)
- [Specify Shipment and Package Details](#)
- [Print Labels and Shipping Documents](#)

Create Shipping Items

In NetSuite, a shipping item is a delivery method for a shipping carrier. It describes how to ship an item and can include shipping rate information, handling rates, and rules for shipping and handling. With a Shipping Partner plug-in implementation, you create shipping items for the shipping partner. You must create shipping items for the shipping partner to select the shipping partner as the carrier on sales orders and item fulfillments.

When you create or edit a shipping item, you select the shipping partner as the carrier and select the service group and service to use as the shipping method. The plug-in implementation defines the service groups and related services that are available to the shipping partner. The values defined by the plug-in implementation appear on the **Shipping Item** page in NetSuite when you create or edit a shipping item. You can also specify the shipping and return label integration properties.

To view existing shipping items or create a new shipping item, go to Lists > Accounting > Shipping Items.

Rate Integration

You can select the rate properties for a shipping partner shipping item on the **Shipping Rate** subtab of the **Shipping Item** page. The following figure shows the rate integration properties on a shipping item:

The following table lists the rate integration properties for a shipping partner:

Property	Description
Shipping Partner	Name of the shipping partner. The values for this property are generated from the list of Shipping Partner plug-in implementations listed on the Manage Plug-ins page. For more information, see Activate the Plug-in .
Service Group	Name of the service group for the shipping item. The plug-in implementation defines the list of service groups for the shipping partner. The plug-in implementation can define multiple service groups for a shipping partner.
Service Name	Name of the service associated with the service group for the shipping item. The plug-in implementation defines the list of services that belong to each parent service group. The plug-in implementation can define multiple services for each service group.
Registration	Name of the partner registration. You must register the shipping partner external shipping account to calculate real-time shipping rates. For more information, see Register a Shipping Partner .

Shipping and Return Label Integration

You select the shipping and return label properties for a shipping partner shipping item on the **Shipping Labels** subtab of the **Shipping Item** page. If this option is enabled, you can print the shipping label from an item fulfillment.

The following figure shows the shipping and return label integration properties for a shipping partner on a shipping item:

The screenshot shows the 'Shipping Labels' configuration page in NetSuite. The 'Shipping Partner Labels' section is highlighted with a red box. It contains the following fields:

- Shipping Partner:** A dropdown menu with 'Royal Mail' selected.
- Service Group:** A dropdown menu with 'Royal Mail UK Letter Services' selected.
- Shipping Label Integration:** A dropdown menu with 'Royal Mail 1st Class Mail' selected. To its right is a 'Registration' checkbox.
- Return Label Integration:** A dropdown menu with 'Royal Mail 2nd Class Mail' selected. To its right is a 'Registration' checkbox.

At the bottom of the highlighted section are buttons for 'Save', 'Cancel', 'Reset', and 'Account'.

The following table lists the rate integration properties for a shipping partner:

Property	Description
Shipping Partner	Name of the shipping partner. The values for this property are generated from the list of Shipping Partner plug-in implementations listed on the Manage Plug-ins page. For more information, see Activate the Plug-in .
Service Name	Name of the service associated with the service group for the shipping item. The plug-in implementation defines how the shipping and return labels are generated for each service.
Registration	Name of the partner registration. You must register the shipping partner external shipping account to use the external shipping partner account to generate shipping and return labels. For more information, see Register a Shipping Partner .

Register a Shipping Partner

A Shipping Partner Registration contains the external shipping account account credentials and other related metadata required by the external account. Use the Shipping Partner Registration form to register the external shipping account in NetSuite. The plug-in implementation uses these account credentials to validate the plug-in implementation with the external account when you calculate real-time shipping rates or generate shipping and return labels.

The Shipping Partner feature adds the **Partner Registration** subtab to the Set Up Shipping page at Setup > Accounting > Shipping. Use this tab to register the shipping partner external account. Shipping Partner registration can be required, optional, or not required, depending on the requirements of the shipping partner:

- Required. The rates are provided by an external partner and are not available without a registered external shipping account.
- Optional. The rates are provided by an external partner but the partner may also provide common rates if the external shipping account is not available.
- Not required. Only flat rates or list rates are provided for the shipping items set up for the shipping partner.

The following figure shows the **Partner Registration** subtab:

The screenshot shows the 'Partner Registration' tab in the NetSuite interface. At the top, there are tabs for 'Carrier Registration', 'Partner Registration' (selected), 'Preferences', and 'Packages'. Below the tabs, there is a 'View' dropdown set to 'Shipping Partner Registrations Default'. Two buttons, 'Add Shipping Partner Registration' and 'Customize View', are visible. A table lists shipping partners with columns for 'SHIPPING PARTNER', 'NAME', and 'DESCRIPTION'. The table contains two entries: 'Pacejet' with 'Pacejet - UPS' and 'Royal Mail' with 'Royal Mail Parcel'. A 'Submit' button is at the bottom left.

SHIPPING PARTNER	NAME	DESCRIPTION
Pacejet	Pacejet - UPS	Pacejet - UPS
Royal Mail	Royal Mail Parcel	Royal Mail Parcel

To register a shipping partner, click **Add Shipping Partner Registration**. Each shipping partner and service group has an associated Shipping Partner Registration form. You can use separate forms for each service group or use the same form for all service groups. A developer creates the form and defines the behavior of the plug-in implementation when you submit the form. The user credentials for the external account are stored in NetSuite.

The associated custom forms are included with the plug-in implementation bundle and are installed by an administrator.

The following figure shows a sample partner registration form:

The screenshot shows the 'Shipping Partner Registration' form. At the top, there are buttons for 'Save', 'Cancel', and 'Reset'. Below these is a 'Service Group' section with a dropdown menu set to 'Royal Mail UK Parcel Services'. The 'Registration Details' section contains several input fields: 'Name*', 'Description', 'First Name', 'Last Name', 'Company', 'Phone', 'Email', 'Address Line 1', 'Address Line 2', 'Address Line 3', 'City', 'State', 'Zip Code', and 'Country'. The 'Credential Fields' section at the bottom has 'User Name*' and 'Password*' input fields. At the very bottom, there are 'Save', 'Cancel', and 'Reset' buttons.

Calculate Shipping Rates

When you calculate shipping rates for a sales order or item fulfillment, NetSuite sends the details for the transaction to the plug-in implementation associated with the selected shipping method for the transaction. The plug-in implementation then calculates the shipping rate and returns the value to NetSuite. NetSuite displays the shipping rate in the **Shipping Cost** field on the **Shipping** subtab for the transaction and in the Summary panel.

The following figure shows the **Shipping** subtab for a sales order. Click the **Calculate** button to calculate shipping rates.

The screenshot shows the NetSuite Shipping subtab interface. It features a top navigation bar with tabs: Items, Shipping, Billing, Accounting, Relationships, and Communication. The main content area is split into two columns. The left column, titled 'Shipping Information', contains fields for 'Ship Date' (6/3/2014), 'Shipping Carrier' (FedEx/USPS/More), 'Shipping Method' (CPS - Business Letter), and 'Shipping Cost' (3.11). The right column, titled 'Shipping Address', contains 'Ship To Select' (default bill address) and 'Ship To' (Harriott Construction Services, 8338 Mission Viejo Circle, Enid, OK 73701). A red rectangular box highlights the 'Shipping Cost' field. Below the form are buttons for 'Save', 'Cancel', 'Reset', and 'Actions'.

Note: You must recalculate rates after you make any change to the transaction that requires a change in the shipping rate. You must also recalculate rates if the rates have changed since the last time the rates were calculated for the transaction.

Specify Shipment and Package Details

For item fulfillments, you can specify additional details about the shipment and the packages for the items in the transaction. The forms that appear are customized by a developer and provided with the plug-in implementation bundle installed by an administrator. The plug-in implementation defines names of the custom forms for the service group selected in the shipping item for the transaction.

Note: The Shipping Label Integration feature must be enabled to specify shipment and package details. Enable the feature at Setup > Company > Enable Features > Transactions.

The following figure shows the **Shipment Details** and **Package Details** buttons for an item fulfillment:

Shipment Details

Use shipment details to gather additional details about the shipment on a transaction level. The **Shipment Details** button is available from the main page of an item fulfillment.

The following figure shows a sample **Shipment Details** popup for an item fulfillment:

Package Details

Use package details to gather additional details each package in a transaction. The **Package Details** button is available from the **Packages** subtab of an item fulfillment. You can specify package details for each package in an item fulfillment.

The following figure shows a sample **Package Details** popup for an item fulfillment:

A screenshot of a 'Package Details' dialog box. At the top are 'Cancel' and 'OK' buttons. Below is a section titled 'Package Details' containing five input fields: 'Description', 'Declared Value', 'Length', 'Height', and 'Width'.

Print Labels and Shipping Documents

Use the Print icon on an item fulfillment record in View mode to print shipping labels and shipping documents as PDFs. For example, these documents can include reports for hazardous materials or documents for international transport. The labels and documents that are available depend on the settings for the shipping item and the definition of the plug-in implementation.

You can select Print > Labels or Print > Shipping Partner Documents.

A screenshot of the NetSuite 'Item Fulfillment' record for '1Harriott Construction Services'. The record number is 39. The page includes buttons for 'Edit', 'Back', and 'Bill'. A red box highlights the 'Actions' menu, which contains the following options: 'Print', 'Print Label', and 'Print Shipping Partner Documents'. Below the actions menu, the 'Primary Information' section shows 'Ref. No. 39', 'Customer 1Harriott Construction Services', and 'Created From Sales Order #182'. The 'Shipment Details' section is partially visible at the bottom.

Developing a Shipping Partner Plug-in Implementation

NetSuite provides a scriptable interface to the Shipping Partner Plug-in. You can create an implementation of the interface that represents a shipping partner in NetSuite. Use the plug-in implementation to create shipping items for the shipping partner.

To create a plug-in implementation, use a developer account to develop and test the plug-in implementation. Then, use SuiteBundler to bundle the plug-in objects and distribute them to other NetSuite accounts. NetSuite administrators use the bundle to install the plug-in implementation and the associated objects in a NetSuite account and enable the shipping partner functionality. For more information about administration tasks for a Shipping Partner Plug-in bundle, see [Administering a Shipping Partner Plug-in Implementation](#).

The following table describes the basic steps in developing a plug-in implementation of the Shipping Partner plug-in:

Step	Description
Enable features	Enable the features required for developing the plug-in implementation, including the Shipping Partner and Server SuiteScript features. For information, see Step 1. Enable Features for Development.
Customize forms	Customize the forms used by the plug-in implementation to register external shipping partner accounts and gather additional shipment and package details on item fulfillments. For information, see Step 2. Customize Shipping Partner Entry Forms.
Create custom record types	Create custom record types for each external shipping partner account that you want to register as a shipping partner. For information, see Step 3. Create Custom Record Type.
Create script file	Create the script file that contains the Shipping Partner plug-in implementation. For information, see Step 4. Create the Plug-in Implementation Script File.  Note: You can also define additional custom data to pass from NetSuite to the plug-in implementation. For information, see Understanding Custom Data in a Shipping Partner Plug-in Implementation.
Add and activate the plug-in implementation	Add the plug-in implementation using the plug-in script file and any required utility files that you created in the previous step to the development account. Then, activate the plug-in implementation. For information, see Step 5. Add and Activate the Plug-in Implementation.
Bundle the plug-in implementation	Bundle the plug-in implementation for distribution to other NetSuite accounts. NetSuite administrators use this bundle to install the plug-in implementation and plug-in script and utility files. For information, see Step 6. Bundle the Plug-in Implementation.

 **Note:** For testing purposes, you need to register the shipping partner and create shipping items. For more information, see [Register Shipping Partner](#) and [Create Shipping Items](#).

Step 1. Enable Features for Development

To begin development of a Shipping Partners plug-in, make sure that the Server SuiteScript feature is enabled on the development account.

To enable features for the Shipping Partner plug-in:

1. Choose Setup > Company > Enable Features.

2. On the **SuiteCloud** subtab, make sure **Server SuiteScript** is checked. If necessary, check the box and agree to the Terms of Service.
3. Click **Save**.

Step 2. Customize Shipping Partner Entry Forms

The Shipping Partner plug-in includes entry forms that you can customize, depending on the plug-in implementation requirements. Go to Customization > Forms > Entry forms to view and customize the entry forms.

Note: If you do not customize a form, the default form appears for all service groups in a shipping partner plug-in implementation. For more information about customizing forms, see the help topic [Creating Custom Entry and Transaction Forms](#) in the Customization documentation.

The following table lists the Shipping Partner entry forms:

Form Name	Description
Shipping Partner Registration Form	Specifies the shipping partner name, service group, and the user credentials used to validate the external partner shipping account. See Shipping Partner Registration Form .
Shipping Partner Shipment Form	Used to collect additional details about the shipment on a transaction level. By default, this form contains a Description field. You add this form to a service group in the plug-in implementation. You can customize multiple versions of this form and add them to the respective service groups. The form that appears on an item fulfillment depends on the service group for the shipping method selected for the transaction.
Shipping Partner Package Form	Used to collect additional details about a specific package on the Packages subtab of an item fulfillment. By default, this form includes the following fields: ■ Description ■ Declared Value ■ Length ■ Width ■ Height You add this form to a service group in the plug-in implementation. You can customize multiple versions of this form and add them to the respective service groups in the plug-in implementation. The form that appears on an item fulfillment depends on the service group for the shipping method selected for the transaction.

Shipping Partner Registration Form

You must use this form to collect any properties required by the external shipping partner account. These values are stored in a custom record in NetSuite and are used when the plug-in implementation needs to request information from the external account, for example, to calculate real-time rates or generate shipping or return labels.

You can create multiple registration forms for each service group in a plug-in implementation. You will require the field names and form name when you develop the plug-in implementation.

The following table describes the default sections of the record form:

Section Name	Description
Service Group	Partner name and service group to register with the external account. Each service group created by the plug-in implementation appears in the Service Group dropdown when you register a shipping partner account.
Registration Details	Contact details for the service group used to ship orders. This information appears as the mailing address when you ship items using the partner account. Name is required.
Credential Fields	User name and password for the external shipping account. Both fields are required. Add any additional fields required by the external account to this section.

For more information about using the form properties and values in a plug-in implementation, see the following topics:

- [ServiceGroup](#)
- [displayRegistration\(DisplayRegistrationInput input, DisplayRegistrationOutput output\)](#)
- [processRegistration\(ProcessRegistrationInput input, ProcessRegistrationOutput output\)](#)

Step 3. Create Custom Record Type

If you want to register external shipping partner accounts to calculate real-time shipping rates or generate shipping and return labels and shipping documents, you must create a custom record type for the external account. NetSuite uses these record types to store the external account user credentials.

For more information about creating custom record types, see the help topic [Creating Custom Record Types](#) in the Customization documentation.

If you need to register multiple external accounts and the accounts require different properties, you may need to create a separate custom record type for each external account. For each record, you will need the following information to define the registration process in the plug-in implementation:

- **Record name.** You must specify the record type name in which to store the user credentials.
- **Field names.** Each property required by the external shipping partner account must have a field in the custom record type. User Name and Password fields are required.

After you create the custom record type in NetSuite, you can use the SuiteScript API functions to manipulate instances of the custom record type in the plug-in implementation. You can use [nlapiCreateRecord\(type, initializeValues\)](#) to create an instance of the custom record type and use [nlapiLoadRecord\(type, id, initializeValues\)](#) and [nlapiSubmitRecord\(record, doSourcing, ignoreMandatoryFields\)](#) to load and submit the record to NetSuite.

Step 4. Create the Plug-in Implementation Script File

You must implement each Shipping Partner plug-in interface function in a Javascript file (with a .js extension) to define the behavior of the plug-in implementation. You can use the SuiteCloud IDE or another Javascript IDE or text editor to create the plug-in script file.

The following sample shows the functions that must be implemented in the plug-in implementation script file:

```

1  function getServiceGroups(input, output)
2  {
3      // define service groups and services
4  }
5  function displayRegistration(input, output)
6  {
7      // display the shipping partner registration credentials
8  }
9  function processRegistration(input, output)
10 {
11     // process partner registration form and store credentials
12 }
13 function calculateRates(input, output)
14 {
15     // calculate shipping rates on a sales order or item fulfillment
16 }
17 function processShipment(input, output)
18 {
19     // process a completed item fulfillment and define shipping documents and labels
20 }
21 function voidShipment
22 {
23     // functionality for handling the deletion of an item fulfillment
24 }

```

Interface Functions

The following table describes the Shipping Partner interface functions:

Function Name	Description
<code>getServiceGroups(GetServiceGroupsInput input, GetServiceGroupsOutput output)</code>	Defines the service groups and services for the shipping partner. Includes the names of the custom entry forms associated with the partner registration, shipment details, and package details.
<code>displayRegistration(DisplayRegistrationInput input, DisplayRegistrationOutput output)</code>	Displays the shipping partner registration credentials on the Shipping Partner Registration page.
<code>processRegistration(ProcessRegistrationInput input, ProcessRegistrationOutput output)</code>	Stores the credential fields for an external shipping partner account entered on the Shipping Partner Registration.
<code>calculateRates(CalculateRatesInput input, CalculateRatesOutput output)</code>	Defines the Shipping Partner plug-in implementation functionality when a user clicks Calculate on a sales order or item fulfillment or saves an item fulfillment. Use this function to communicate with the external shipping partner account and return the rate calculation for a sales order or item fulfillment.
<code>processShipment(ProcessShipmentInput input, ProcessShipmentOutput output)</code>	Defines the Shipping Partner plug-in implementation functionality when a user saves an item fulfillment or submits a Web store order. Use this function to communicate with the external shipping partner account to process the item fulfillment or Web store order.
<code>voidShipment(VoidShipmentInput input, VoidShipmentOutput output)</code>	Defines the Shipping Partner plug-in implementation functionality when a user deletes an item fulfillment or Web store order. This function provides access the transaction key for the transaction so the shipping partner can process the cancellation.

Function Name	Description
	Use this function to communicate with the external shipping partner account to process the deletion of the transaction.

Rules and Guidelines for Creating a Plug-in Implementation Script File

Use the following rules and guidelines when creating the plug-in implementation script file:

- The plug-in script file can have any name, as long as it contains an implementation of each of the interface functions.
- If you want to create utility files with helper functions to use with the plug-in implementation script file, you can include those files when you create the plug-in implementation in NetSuite. See [Step 5. Add and Activate the Plug-in Implementation](#).
- Due to limitations with how strings are handled in NetSuite, you cannot use the JavaScript case statement in the plug-in implementation script file.
- The Shipping Partner plug-in allows up to 1000 usage units when used with SuiteScript. See the help topic [SuiteScript Governance and Limits](#).

Understanding Custom Data in a Shipping Partner Plug-in Implementation

You can use field values on an item fulfillment or Web store order as custom data in a shipping partner plug-in implementation. You can direct NetSuite to send specific field values as custom data to the plug-in implementation and then use the custom data to calculate rates and process the transaction. During rate calculation and shipment processing, you can also set certain custom data field values.

Although you can use the SuiteScript API [nlapiLoadRecord\(type, id, initializeValues\)](#) to access and set field values in a transaction, use custom data to increase the performance of the plug-in implementation. Using custom data to send only the field values you need to access, rather than loading the entire transaction record or the entire shipment, item, or package record.

The type of custom data that you can retrieve and set depends on the type of plug-in functionality you implement. The following table describes the plug-in functionality where you can send, retrieve, and set custom data:

Plug-in Functionality	Custom Data Usage
Create service groups	Use the CustomDataDescriptor object to define the shipment, item, and package field values that NetSuite sends to the plug-in implementation. For more information, see Using Custom Data with Service Groups
Calculate rates	Use the methods available to the Shipment , Item , and Package objects to get the custom data field values. You can also use the methods for the Rate object to set custom data field values for the transaction that the plug-in implementation passes back to NetSuite. For more information, see Using Custom Data in Rate Calculation .
Process shipment	Use the methods available to the ProcessShipmentOutput interface output object to access and set shipment and package custom data field values. When you set the custom data values, the plug-in implementation sends these values back to NetSuite. For more information, see Using Custom Data in Shipment Processing .

Note: Make sure that you review the guidelines for using custom data. For information, see the help topic [Custom Data Rules and Guidelines](#).

Using Custom Data with Service Groups

You must define the custom data that NetSuite sends to the plug-in implementation. The custom data is defined for each service group in the plug-in implementation. When a user calculates rates or saves an item fulfillment or Web store order, NetSuite sends the custom data defined by the plug-in implementation for the service group selected for the transaction.

You must create a [CustomDataDescriptor](#) object and then use the [addItemParameter\(parameter,sourceField\)](#), [addPackageParameter\(parameter,sourceField\)](#), or [addShipmentParameter\(parameter,sourceField\)](#) methods to define the custom data field values to send to the plug-in implementation.

For each parameter you add to the service group as custom data, NetSuite creates a key-value pair with the implementation-defined parameter name as the key and the field value as the value. For example, if you add two shipment fields as parameters to a service group, NetSuite creates a key-value pair for each parameter. You can then access the values when you calculate rates or process a shipment.

The following example shows the **Memo** field and the **Description** field from the Shipment Details entry form that are added to the lettergroup service group:

```
1 customDataDescriptor = output.getObjectFactory.newCustomDataDescriptor();
2 customDataDescriptor.addShipmentParameter('txn_memo', 'memo')
3 customDataDescriptor.addShipmentParameter('txn_desc', 'shipmentdetails.description')
4 letterGroup.setCustomDataDescriptor(customDataDescriptor);
```

If you want to access values for `txn_memo` or `txn_desc` when you calculate rates or process the shipment, you access the value with the parameter name as the key. You can also retrieve all custom data key-value pairs per item, shipment, or package.

The following figure shows the location of the **Memo** and **Description** fields on an item fulfillment from the previous example:

Using Custom Data in Rate Calculation

After you add custom data parameters to a service group, you can access those values when you calculate rates. Use the methods available to the [Shipment](#), [Item](#), and [Package](#) objects to retrieve the custom data values. In addition, the [Rate](#) object allows you to set transaction-level field values. These custom data values are passed back to NetSuite from the plug-in implementation. You cannot set custom data field values available on the [Shipment Details](#) or [Package Details](#) entry forms.

For additional guidelines for using custom data in rate calculation, see [Custom Data Rules and Guidelines](#).

The following example shows how to access a custom data value on a shipment named `txn_desc` and set a custom field value for `custbodyshipa_rm_deliverydate` on the `Rate` object:

```
1 var desc = shipment.getCustomData("txn_desc");
2 ...
3 rate.getCustomData().setAttribute('custbodyshipa_rm_deliverydate', deliveryDate.toString());
```

In the example above, the key `custbodyshipa_rm_deliverydate` denotes a field that corresponds to the **Delivery Date** field on an item fulfillment. The following figure shows the field value set by the [setAttribute\(key,value\)](#) method for a [CustomData](#) object:

Using Custom Data in Shipment Processing

After you add custom data parameters to a service group, you can access those values when you process a shipment. You can access custom data field values for the fields available through the [Shipment Details](#)

or [Package Details](#) entry forms that were defined by [CustomDataDescriptor](#). Use the methods available to the [CustomDataContainer](#) object to retrieve or set custom data values. Any custom data field values that you set are passed back to NetSuite by the plug-in implementation.

The CustomDataContainer object contains methods to return all custom data for a shipment or package as [CustomData](#) objects. Use the CustomData methods to retrieve or set custom data values.

The following example shows how to retrieve the value for the txn_desc parameter on the shipment and set the value for the custrecordshipa_rm_output custom field on a package:

```
1 // access custom data
2 shipment.getCustomData("txn_desc");
3 ...
4 // set custom data
5 var newData = getObjectFactory.newCustomData();
6 newData.setAttribute('custrecordshipa_rm_output', packageResults);
7 output.getCustomDataContainer().setPackageCustomData(pkg.getId(), newData);
```

Custom Data Rules and Guidelines

Use the following rules and guidelines with custom data in a Shipping Partner plug-in implementation:

- If you have the Multiple Shipping Routes feature enabled in the account, you cannot use custom data in a Shipping Partner plug-in implementation.
- For security reasons, you cannot use custom data for sales order transactions. Any methods that retrieve custom data return a null value if the transaction is a sales order.
- With the exception of the trackingnumber or trackingnumberurl fields, you can only set custom data field values for custom fields. The plug-in implementation returns an error if you attempt to set the value of a standard NetSuite transaction field.

Step 5. Add and Activate the Plug-in Implementation

You must create a Shipping Partner plug-in implementation with the script file and any utility files and then activate the implementation. After you complete this step, the plug-in implementation appears as a shipping partner on the **Set Up Shipping** and **Shipping Item** pages in NetSuite.

To add the plug-in implementation:

1. In NetSuite, go to Customization > Plug-ins > Plug-in Implementations > New.
2. On the **Select Plug-in Type** page, click **Shipping Partners**.
3. On the **Plug-in Implementation** page, enter the following information:

Option	Description
Name	User-friendly name for the implementation. This name appears as the shipping partner in the following locations: ■ Manage Plug-ins page. Page used by administrators to enable/disable the implementations in their account. ■ Set Up Shipping page. ■ Shipping Partner Registration page. ■ Shipping Labels subtab on the Shipping Items page. ■ Bundle Builder
ID	Internal ID for the implementation for use in scripting. If you do not provide an ID, NetSuite will provide one for you after you click Save. As a developer, when you write code that uses this implementation, you will reference the implementation using this ID.

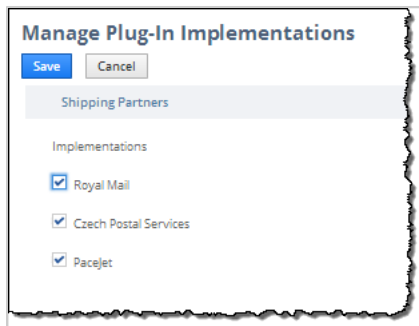
Option	Description
Status	Current status for the implementation. Choose Testing to have the implementation accessible to the owner of the implementation. Choose Released to have the implementation accessible in a production environment
Log Level	Logging level you want for the script. Select Debug, Audit, Error, or Emergency. The log information appears on the Execution Log subtab for the implementation after you create it. Go to Customization > Plug-ins > Plug-in Implementations, select your implementation, and then click the Execution Log subtab.
Description	Optional description of the implementation. The description appears for the implementation on the Shipping Partner Registrations page.
Owner	User account that owns the implementation. Default is the name of the current user

- On the **Scripts** subtab, in the **Implementation** dropdown, select the script file that contains the implementation of the plug-in. Choose **New** to open the File page to select a file.
- On the **Scripts** subtab, in the **Library Script File** list, select any utility script files required by the plug-in implementation script file that you uploaded in step 3.
- On the **Unhandled Errors** subtab, select which individual(s) will be notified if script errors occur. By default the **Notify Script Owner** box is checked.

To enter multiple email addresses in the **Notify Emails** box, separate email addresses with a semi-colon.

- Click **Save**. You can access the list of implementations by going to Customization > Plug-ins > Plug-in Implementations.
- To activate the plug-in implementation, go to Customization > Plug-ins > Manage Plug-ins.

9. Under **Shipping Partners**, check the box next to name of your implementation.



10. Click **Save**.

Step 6. Bundle the Plug-in Implementation

After developing the Shipping Partner plug-in implementation, you can distribute the implementation to a production account. SuiteBundler allows NetSuite users to package together groups of objects for distribution to other accounts. These packages are called bundles, or SuiteApps. To distribute the Shipping Partner plug-in implementation script file and utility files, custom forms and records, and implementation object, create a bundle with SuiteBundler. After you create the bundle, administrators can install the bundle in production accounts.

The following table lists the objects you must include in the bundle and their location on the **Select Objects** page in the Bundle Builder:

Object	Location On Select Objects Page
Plug-in implementation script file	File Cabinet > Files
Utility files	
Custom records	Custom Lists/Records > Records
Custom forms	Forms > Entry Forms
Plug-in implementation	Plug-ins > Shipping Partner Implementations

For more information about creating a bundle, see [Steps for Creating a Bundle in the SuiteBundler documentation](#).

Administering a Shipping Partner Plug-in Implementation

After a developer creates a plug-in and distributes it, you can install and set up the shipping partner.

To install and set up a Shipping Partner plug-in implementation, complete the following steps:

- [Enable Features for Shipping Partner Plug-in](#)
- [Install a Shipping Partner Plug-in Bundle](#)
- [Activate the Plug-in](#)

- [Register Shipping Partner](#)
- [Create Shipping Items](#)

Enable Features for Shipping Partner Plug-in

To begin development of a Shipping Partners plug-in, make sure that the Server SuiteScript feature is enabled on the development account.

To enable features for the Shipping Partner plug-in:

1. Choose Setup > Company > Enable Features.
2. On the **SuiteCloud** subtab, make sure the Server SuiteScript is checked. If necessary, check the box and agree to the Terms of Service.
3. Click **Save**.

Install a Shipping Partner Plug-in Bundle

After a developer creates an implementation of a Shipping Partner plug-in and bundles it, you can install and set up the plug-in implementation bundle.

To install a Shipping Partner plug-in bundle:

1. Go to Customization > SuiteBundler > Search & Install Bundles.
2. On the Install Bundle page, choose Production Account in the Location dropdown.
3. Search for the plug-in bundle.
4. Click Install for the bundle.



Important: To avoid duplicate objects during install, select “Replace Existing Object” if prompted.

After you begin the installation of a bundle, you can continue working in NetSuite as the bundle installs.

To check on the progress of the installation, go to the list of installed bundles at Customization > SuiteBundler > Search & Install Bundles > List. If installation is not complete, the Status column displays the percentage of installation progress. Click Refresh to update the status. When installation is complete, the Status column displays a green check.



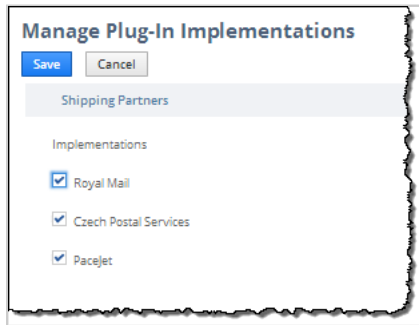
Note: If no Install button is available, this bundle may not have been shared with your account. To get access to the bundle, contact the developer or NetSuite Technical Support.

Activate the Plug-in

To enable the Shipping Partner plug-in implementation and create shipping items for the shipping partner, you must activate the implementation.

To activate the Shipping Partners plug-in:

1. In NetSuite, go to Customization > Plug-ins > Manage Plug-ins.
2. Under **Shipping Partners**, check the box next to the name of the plug-in implementation.



3. Click **Save**.

Register Shipping Partner

A Shipping Partner Registration contains the external shipping account credentials. Use the Shipping Partner Registration form to register a service group with an external shipping account to calculate shipping rates and generate shipping and return labels and shipping documents.

Note: To register a shipping partner, at least one service group in the plug-in implementation must with the registration type set to MANDATORY or OPTIONAL.

To register a shipping partner:

1. Go to Setup > Accounting > Shipping.
2. Click the **Partner Registration** subtab. The subtab lists all currently enabled shipping partner implementations.
3. Click **Add Shipping Partner Registration**.
4. On the **Select Shipping Partner** page, select the shipping partner.
5. On the **Shipping Partner Registration** page, enter the required information.

The following fields are required:

- **Service Group** under Service Group.
- **Name** field under Registration Details.
- **User Name** and **Password** under Credential Fields.

The other information on the page depends on the customization of the Shipping Partner Registration form.

6. Click **Save**.

Create Shipping Items

After you activate the plug-in implementation, you can create a shipping items to test the plug-in implementation functionality. When you create the shipping item, you select a service group and service defined by the plug-in implementation for the shipping rate and labels.

For more information about shipping items, see the help topic [Setting Up Shipping](#) in the Order Management documentation.

To create a shipping item:

1. In NetSuite, go to Lists > Accounting > Shipping Items > New.
2. On the **Shipping Item** page, enter the following shipping item properties:

Property	Description
Ship Name	Name for the shipping item. This property appears as the Shipping Method on the Shipping subtab of a sales order or item fulfillment or as a ship method in a Web store. This property is required.
Display Name/Code	External name for the item that appears in the Shipping Method or Ship Via field on printed forms, your Web store, and all centers, such as the Customer Center. In all other cases, the Ship Name displays for the item. This property is required.
Description	Description of the shipping item.
Subsidiaries (OneWorld only)	Subsidiary for this shipping item. You can create shipping items for specific subsidiaries.
Inactive	Inactivates the shipping item. If you check this box, users can no longer select this shipping item on transactions.
Display in Web Site	Makes the shipping item appear as a shipping method in your Web store.

3. On the **Shipping Rate** subtab, check the **Shipping Partner Rate** box and enter the following shipping item properties:

Property	Description
Account (Shipping)	Account for shipping charges for this shipping item This property is required.
Shipping Partner	Name of the plug-in implementation for the shipping partner.
Service Group	Service group defined by the plug-in implementation that contains the service you want to use for this shipping item.
Service Name	Name of the service for this shipping item.
Registration	Name of the shipping partner registration for the service group that you created on the Shipping Partner Registration page. You must select a registration if you want to calculate real-time shipping rates for this shipping item. If you want to use list or flat rates for this shipping item, this property is not required.

4. If the plug-in implementation contains functionality for shipping and return labels, on the **Shipping Labels** subtab, check the **Shipping Partners** box and enter the following properties:

Property	Description
Shipping Partner	Name of the plug-in implementation for the shipping partner.
Service Group	Name of the service group in the plug-in implementation that contains the functionality for generating labels.
Service Group	Service group defined by the plug-in implementation that contains the service you want to use for this shipping item.

Property	Description
Service Name	Name of the service for this shipping item.
Registration	Name of the shipping partner registration for the service group that you created on the Shipping Partner Registration page. You must select a registration if you want to calculate real-time shipping rates for this shipping item. If you want to use list or flat rates for this shipping item, this property is not required.

Shipping Partner Interface Definition

Interface Functions

The Shipping Partner interface includes the following functions.

Function	Description
getServiceGroups(GetServiceGroupsInput input, GetServiceGroupsOutput output)	Defines the service groups and associated services available to a shipping partner represented by the Shipping Partner plug-in implementation. You must use this function to define all service groups and services for the shipping partner. In addition, use this function to define the custom entry forms for each service group and custom data to pass to the plug-in implementation. The custom entry forms include entry forms for partner registration, shipments, and packages. Create the service groups, services, and entry forms and use the methods available to the interface output object GetServiceGroupsOutput to pass the objects created by this function back to NetSuite.
displayRegistration(DisplayRegistrationInput input, DisplayRegistrationOutput output)	Defines the shipping partner registration credentials that appear on the Shipping Partner Registration page. This function defines the external shipping account credentials that are required to connect to the external shipping partner account to calculate real-time rates and generate shipping documents. This function is called by NetSuite when a user opens a shipping partner registration form. The credentials defined with this function are placed on the shipping partner registration page and stored by the processRegistration(ProcessRegistrationInput input, ProcessRegistrationOutput output) interface function when a user submits the registration form.
processRegistration(ProcessRegistrationInput input, ProcessRegistrationOutput output)	Defines the plug-in implementation functionality when a user submits a shipping partner registration form. This function stores the credential fields for an external shipping partner account entered on the shipping partner registration entry form.
calculateRates(CalculateRatesInput input, CalculateRatesOutput output)	Define the plug-in implementation functionality when a user selects a shipping method for the shipping partner and calculates shipping rates on a sales order, item fulfillment, or Web store order. The interface input object CalculateRatesInput provides the sales order, item fulfillment, and Web store order properties, including all items, packages, address and contact information, and shipping method properties. The input object also includes custom data for the shipment, items, and packages. Use these properties to define shipping partner-specific logic to calculate shipping rates. Use the interface output object CalculateRatesOutput to create the shipping rates and custom data values and pass the rates and custom data back to NetSuite.

Function	Description
<code>processShipment(ProcessShipmentInput input, ProcessShipmentOutput output)</code>	Define the plug-in implementation functionality user saves an item fulfillment. The functionality defined by this function can include partner-specific functionality on the shipping partner account and generation of shipping documents. The input object <code>ProcessShipmentInput</code> provides the sales order, item fulfillment, and Web store order properties, including all items, packages, address and contact information, and shipping method properties. The input object also provides the internal key for the transaction that is generated by NetSuite. The input object also includes custom data for the shipment and packages that you can use when processing the shipment. This function also defines any shipping documents required by the transaction, and passes them back to NetSuite with the <code>ProcessShipmentOutput</code> object.
<code>voidShipment(VoidShipmentInput input, VoidShipmentOutput output)</code>	Defines the plug-in implementation functionality when a user deletes an item fulfillment or Web store order. This function provides access the transaction key for the item fulfillment or Web store order so the shipping partner can process the cancellation. The interface input object <code>VoidShipmentInput</code> provides the NetSuite-generated key for the transaction to define logic that processes the cancellation on the shipping partner external account.



Important: You cannot change these function signatures in a Shipping Partner plug-in implementation.

getServiceGroups(GetServiceGroupsInput input, GetServiceGroupsOutput output)

Function Declaration	<code>void getServiceGroups(GetServiceGroupsInput input, GetServiceGroupsOutput output)</code>
Type	Interface function
Description	Defines the service groups and associated services available to a shipping partner represented by the Shipping Partner plug-in implementation. You must use this function to define all service groups and services for the shipping partner. In addition, use this function to define the custom entry forms for each service group. The custom entry forms include entry forms for partner registration, shipments, and packages. The service groups and services that you define in this function appear in the following locations in NetSuite: ■ Set Up Shipping page. ■ Shipping Partner Registration page. ■ Shipping Rate and Shipping Labels subtabs on the Shipping Items page.

Create the service groups, services, and entry forms and use the methods available to the interface output object [GetServiceGroupsOutput](#) to pass the objects created by this function back to NetSuite.

Returns	void
Parameters	■ GetServiceGroupsInput ■ GetServiceGroupsOutput

GetServiceGroupsInput

Type	Interface input object
Description	Input object for the getServiceGroups(GetServiceGroupsInput input, GetServiceGroupsOutput output) interface function.

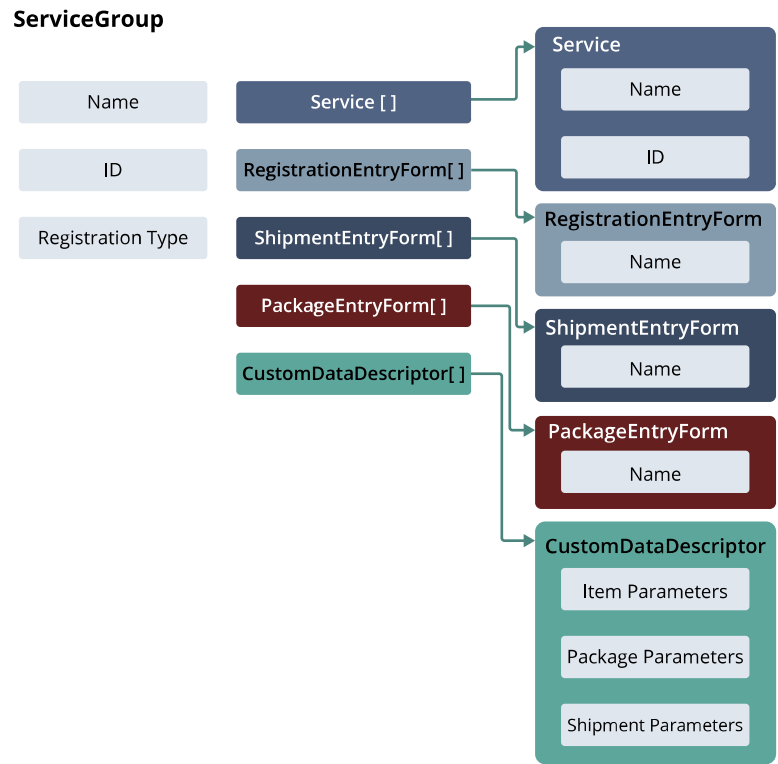
 **Note:** Currently, this object does not pass any transaction properties to the plug-in implementation.

Methods	getLocale()
Child Objects	n/a

GetServiceGroupsOutput

Type	Interface output object
Description	Output object for the getServiceGroups(GetServiceGroupsInput input, GetServiceGroupsOutput output) interface function.
Methods	■ getServiceGroupsContainer() ■ getNotificationsContainer() ■ getObjectFactory()
Child Objects	■ ServiceGroupsContainer ■ NotificationsContainer

GetServiceGroupsOutput Object Model



The following figure shows the GetServiceGroupsOutput object model:

getServiceGroupsContainer()

Type	Object method
Returns	ServiceGroupsContainer
Description	Returns a ServiceGroupsContainer object that contains all service groups defined by the plug-in implementation. Returns an empty string if no service groups exist. Use this method if you want to access all service groups or if you want to add a service group to ServiceGroupsContainer with addServiceGroup(serviceGroup). For more information about service groups, see ServiceGroup.
Parent object	GetServiceGroupsOutput

Example

```
1 function getServiceGroups(GetServiceGroupsInput input, GetServiceGroupsOutput output)
2 {
3     ...
4
5     // get all service groups
6     var container = output.getServiceGroupsContainer();
7
8     ...
9
10    // add service group 'lettergroup' to ServiceGroupsContainer
```

```

11 container.addServiceGroup(letterGroup);
12
13 ...
14 }

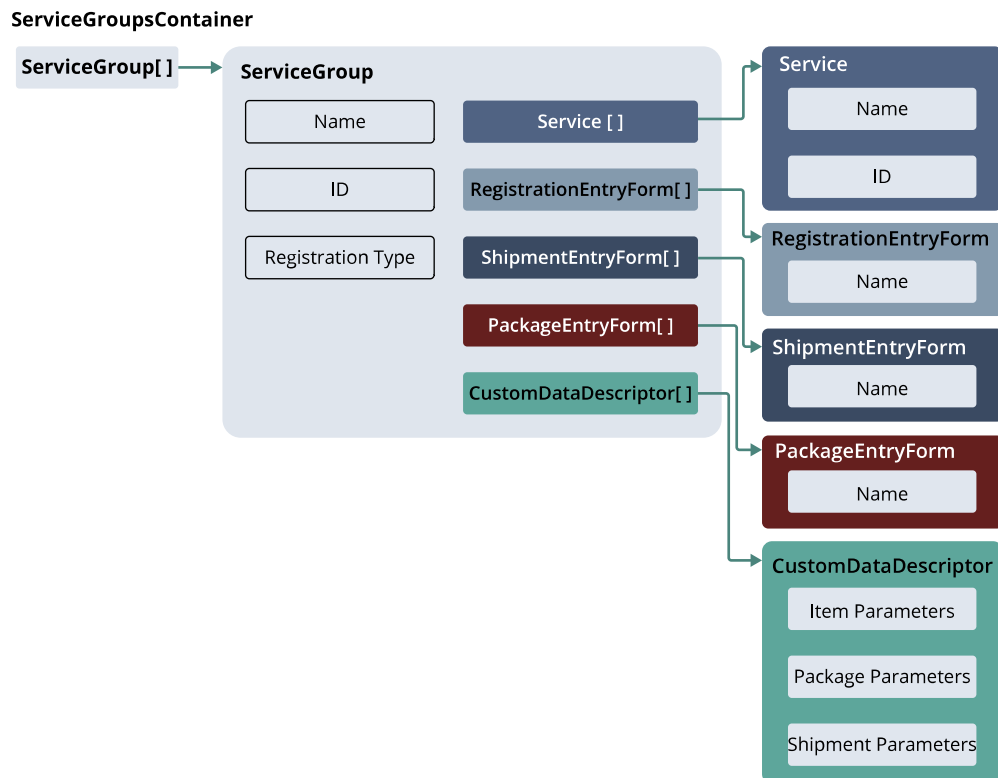
```

ServiceGroupsContainer

Type	Object
Description	The ServiceGroupsContainer object includes all service groups defined by the plug-in. Use this object to add a service group to the plug-in using addServiceGroup(serviceGroup) or use this object to access all service groups with getServiceGroups() .
Methods	■ addServiceGroup(serviceGroup) ■ getServiceGroups()
Parent Object(s)	GetServiceGroupsOutput
Child Object(s)	ServiceGroup

ServiceGroupsContainer Object Model

The following figure shows the ServiceGroupsContainer object model:



addServiceGroup(serviceGroup)

Type	Object method
------	---------------

Function Declaration	<code>void addServiceGroup(ServiceGroup serviceGroup)</code>
Description	Add a service group to the array of service groups included in the plug-in implementation. Before you call this function, you must create a service group using newServiceGroup(id). Each service group must have the name defined for the service group using setName(name) or the method fails.
Returns	<code>void</code>
Input Parameter(s)	<code>serviceGroup {ServiceGroup}</code> — Service group created with newServiceGroup(id) .
Parent object	ServiceGroupsContainer

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     // add service groups 'lettergroup' and 'parcelGroup' to ServiceGroupsContainer
5     output.getServiceGroupsContainer().addServiceGroup(letterGroup);
6     output.getServiceGroupsContainer().addServiceGroup(parcelGroup);
7     ...
8 }
```

getServiceGroups()

Type	Object method
Function Declaration	<code>ServiceGroup[] getServiceGroups()</code>
Description	Returns an array of all ServiceGroup objects defined by the plug-in implementation and added to ServiceGroupsContainer . This method returns an empty array if there are no service groups defined. Use this method to access individual service groups.
Returns	ServiceGroup[]
Input Parameters	None.
Parent object	ServiceGroupsContainer

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     var serviceGroups = output.getServiceGroupsContainer().getServiceGroups();
5
6     for (var i = 0; i < serviceGroups.length; i++)
7     {
8         var currGroup = serviceGroups[i];
9         // process one group here
10    }
11    ...
12 }
```

ServiceGroup

Type	Object
-------------	--------

Description A service group is a set of shipping services defined for a shipping partner. You can define multiple service groups for a single shipping partner. For example, a shipping partner may want a group of services specifically for processing letters and another for processing packages. The service groups defined in the plug-in implementation appear on the **Shipping Rate** and **Shipping Labels** subtabs when you create a new shipping item in NetSuite.

To get started with a service group, create a new service group with [newServiceGroup\(id\)](#) and then define the service group properties with the methods available to the ServiceGroup object, including custom entry forms and custom data.

You can use the ServiceGroup methods to set and retrieve the following properties of a service group:

- Services. Add [Service](#) objects.
- Custom data. Add a [CustomDataDescriptor](#) object.
- Entry forms. Includes shipping partner registration, package entry, and shipment entry forms.
- Registration type.
- Name and id.

After you define the properties, add the service group to the list of service groups defined by the plug-in with [addServiceGroup\(serviceGroup\)](#).

Methods

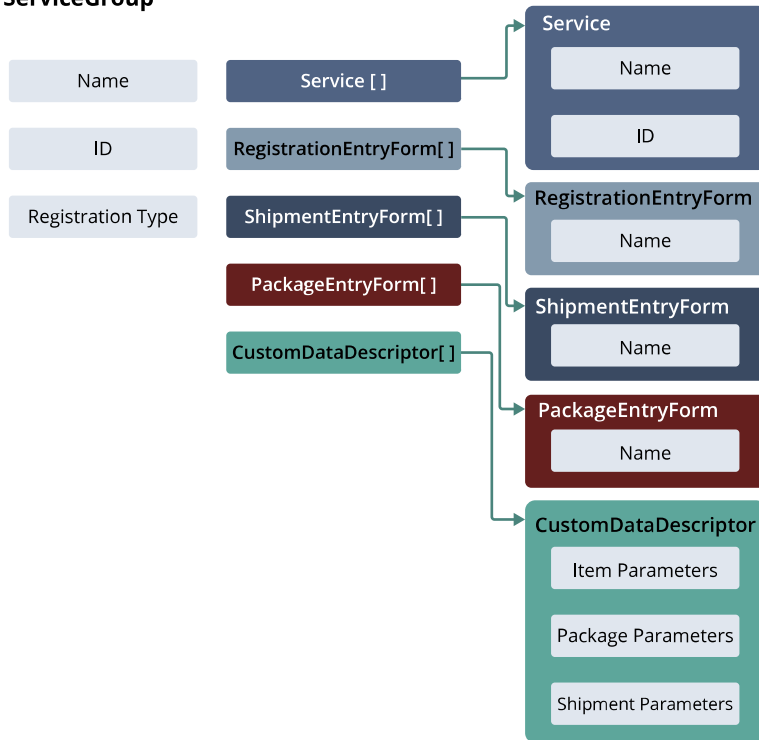
- [addPackageEntryForm\(entryForm\)](#)
- [addRegistrationEntryForm\(entryForm\)](#)
- [addService\(service\)](#)
- [addShipmentEntryForm\(entryForm\)](#)
- [getCustomDataDescriptor\(\)](#)
- [getID\(\)](#)
- [getName\(\)](#)
- [getPackageEntryForms\(\)](#)
- [getRegistrationEntryForms\(\)](#)
- [getRegistrationType\(\)](#)
- [getServices\(\)](#)
- [getShipmentEntryForms\(\)](#)
- [setCustomDataDescriptor\(customDataDescriptor\)](#)
- [setId\(id\)](#)
- [setName\(name\)](#)
- [setRegistrationType\(registrationType\)](#)

Parent Object(s) [ServiceGroupsContainer](#)

Child Object(s) [CustomDataDescriptor](#), [Service](#)

ServiceGroup Object Model

The following figure shows the ServiceGroup object model:

ServiceGroup**addPackageEntryForm(entryForm)****Type** Object method**Function Declaration** void addPackageEntryForm(string entryForm)**Description** Associates the name of a custom package entry form with a [ServiceGroup](#). A custom entry form with this name must exist in the account where you enable the plug-in implementation. You can associate multiple package entry forms with a single service group.You must create the service group using [newServiceGroup\(id\)](#) before you can add the form.This form appears in NetSuite when you select a shipping method that is associated with the service group on a sales order or item fulfillment and then click the **Package** icon under **Package Details** on the **Packages** subtab.For more information about creating the custom entry forms for the Shipping Partner plug-in implementation, see [Step 2. Customize Shipping Partner Entry Forms](#).**Returns** void**Input Parameters** entryForm {string} — Name of the custom entry form for packages. Must be a legal value or the method returns an error.**Parent object** [ServiceGroup](#)**Example**

```

1 | function getServiceGroups(input, output)
2 | {
3 |     ...

```

```

4      // create the service group
5      var letterGroup = output.getObjectFactory().newServiceGroup('letter');
6      ...
7      // add customized Shipping Partner Package Form
8      letterGroup.addPackageEntryForm('First Class Letter Package Details');
9      ...
10 }

```

addRegistrationEntryForm(entryForm)

Type	Object method
Function Declaration	void addRegistrationEntryForm(string entryForm)
Description	Associates the name of a custom registration entry form with a ServiceGroup. A custom entry form with this name must exist in the account where you enable the plug-in. You can associate multiple registration entry forms with a single service group. You must create the service group using newServiceGroup(id) before you can add the form. In addition, you must use setRegistrationType(registrationType) with a value of Mandatory or Optional for the service group. This form name appears in NetSuite on the Partner Registration subtab when you set up shipping. You use this form to register with an external shipping partner account if you want to set up real-time rate calculation or externally generate shipping documents. For more information about creating the custom entry forms for the Shipping Partner plug-in implementation, see Step 2. Customize Shipping Partner Entry Forms.
Returns	void
Input Parameters	entryForm {string} — Name of the custom entry form for shipping partner registration. Must be a legal value or the method returns an error.
Parent object	ServiceGroup

Example

```

1  function getServiceGroups(input, output)
2  {
3      ...
4      // create the service group
5      var letterGroup = output.getObjectFactory().newServiceGroup('letter');
6      ...
7      // set registration type and add customized Shipping Partner Registration form
8      letterGroup.setRegistrationType('MANDATORY');
9      letterGroup.addRegistrationEntryForm('Letter Shipping Partner Registration');
10     ...
11 }

```

addService(service)

Type	Object method
Function Declaration	void addService(Service service)
Description	Associates a Service object with a ServiceGroup. Use newService(id) to create the Service object and set the service name before you call this method. You can only add a service to one service group.
Returns	void

Input Parameters service {[Service](#)} — Service object to add the associated service group.

Parent object [ServiceGroup](#)

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     // create the service group
5     var letterGroup = output.getObjectFactory().newServiceGroup('letter');
6     ...
7     // create the service, set name, and add to 'letter' service group
8     var firstClass = output.getObjectFactory().newService('firstclass');
9     firstClass.setName('1st Class Mail');
10    letterGroup.addService(firstclass);
11    ...
12 }
```

addShipmentEntryForm(entryForm)

Type Object method

Function Declaration void addShipmentEntryForm(string entryForm)

Description Associates the name of a custom shipment details entry form with a [ServiceGroup](#). A custom entry form with this name must exist in the account where you enable the plug-in. You can associate multiple shipment details entry forms with a single service group.

You must create the service group using [newServiceGroup\(id\)](#) before you can add the form.

This form appears when you select a shipping method that is associated with the service group on a sales order or item fulfillment and click **Shipment Details**.

For more information about creating the custom entry forms for the Shipping Partner plug-in implementation, see [Step 2. Customize Shipping Partner Entry Forms](#).

Returns void

Input Parameters entryForm {string} — Name of the custom entry form for shipment details. Must be a legal value or the method returns an error.

Parent object [ServiceGroup](#)

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     // create the service group
5     var letterGroup = output.getObjectFactory().newServiceGroup('letter');
6     ...
7     // add customized Shipping Partner Shipment Form
8     letterGroup.addShipmentEntryForm('First Class Letter Shipment Details');
9     ...
10 }
```

getID()

Type Object method

Function Declaration	string getId()
Description	Returns the unique ID of a ServiceGroup . You must use newServiceGroup(id) to create the service group before you call this method.
Returns	string
Input Parameters	None.
Parent object	ServiceGroup

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     // create a service group
5     var letterGroup = output.getObjectFactory().newServiceGroup('letter');
6     // get the id
7     var letterGroupId = letterGroup.getId();
8     ...
9 }

```

getCustomDataDescriptor()

Type	Object method
Function Declaration	CustomDataDescriptor getCustomDataDescriptor()
Description	Returns the CustomDataDescriptor object associated with a specific service group. Use this method if you want to access the shipment, package, or item custom data added to the service group with setCustomDataDescriptor(customDataDescriptor). This method returns an empty CustomDataDescriptor if no custom data parameters have been added. For more information about using custom data, see Understanding Custom Data in a Shipping Partner Plug-in Implementation.
Returns	CustomDataDescriptor
Input Parameters	None.
Parent object	ServiceGroup

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     // get all service groups
5     serviceGroups = output.getServiceGroupsContainer().getServiceGroups();
6     for (var i in serviceGroups) // process all groups
7     {
8         thisGroup = serviceGroups[i];
9
10        // get the descriptor
11        var thisDescriptor = thisGroup.getCustomDataDescriptor();
12        ...
13    }
14 }

```

getPackageEntryForms()

Type	Object method
Function Declaration	string[] getPackageEntryForms()
Description	Returns an array of the custom package entry form names that you added to a ServiceGroup with addPackageEntryForm(entryForm). This method returns an empty array if no package entry forms are defined for the service group. For more information about adding package entry forms, see addPackageEntryForm(entryForm). For more information about creating the custom entry forms for the Shipping Partner plug-in implementation, see Step 2. Customize Shipping Partner Entry Forms.
Returns	string[]
Input Parameters	None.
Parent object	ServiceGroup

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     // add package form name to service group
5     letterGroup.addPackageEntryForm('First Class Package');
6     letterGroup.addPackageEntryForm('First Class Package - Alternate');
7     ...
8     // get the names
9     var myPackageForms = letterGroup.getPackageEntryForms();
10    for (var i = 0; i < myPackageForms.length; i++) {
11        ...
12    }

```

getRegistrationEntryForms()

Type	Object method
Function Declaration	string[] getRegistrationEntryForms()
Description	Returns an array of custom package entry form name that you added to a ServiceGroup with addPackageEntryForm(entryForm). This method returns an empty array if no package entry forms are defined for the service group. For more information about adding registration entry forms, see addPackageEntryForm(entryForm). For more information about creating the custom entry forms for the Shipping Partner plug-in implementation, see Step 2. Customize Shipping Partner Entry Forms.
Returns	string[]
Input Parameters	None.
Parent object	ServiceGroup

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     letterGroup.addRegistrationEntryForm('Letter Shipping Partner Registration');
5     letterGroup.addRegistrationEntryForm('Letter Shipping Partner Registration - Alternate');
6     ...
7     // get the names
8     var myRegistrationForms = letterGroup.getRegistrationEntryForms();
9     for (var i = 0; i < myRegistrationForms.length; i++) {
10         var myForm = myRegistrationForms[i];
11         ...
12     }

```

getRegistrationType()

Type	Object method
Function Declaration	string getRegistrationType()
Description	Returns a string that represents the registration type for a ServiceGroup . Possible values are MANDATORY, OPTIONAL, or NONE. This method returns null if the registration type was not set for the service group.
Returns	string NULL
Input Parameters	None.
Parent object	ServiceGroup

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     // set registration type and add registration form
5     letterGroup.setRegistrationType('MANDATORY');
6     letterGroup.addRegistrationEntryForm('Letter Shipping Partner Registration');
7     ...
8     var registrationType = letterGroup.getRegistrationType();
9     ...
10 }

```

getServices()

Type	Object method
Function Declaration	Service[] getServices()
Description	Returns an array of Service objects that represents all services defined by the plug-in for a service group. Add a service to a ServiceGroup with addService(service) before you call this method. This method returns an empty array if no services are defined for the service group.
Returns	Service[]
Input Parameters	None.
Parent object	ServiceGroup

Example

```

1 function getServiceGroups(input, output)

```



```

2  {
3      ...
4      // create the service group
5      var letterGroup = output.getObjectFactory().newServiceGroup('letter');
6      ...
7      // create the services, set names, and add to 'letter' service group
8      var firstClass = output.getObjectFactory().newService('firstclass');
9      var firstClass = output.getObjectFactory().newService('secondclass');
10
11     firstclass.setName('1st Class Mail');
12     secondclass.setName('2nd Class Mail');
13
14     letterGroup.addService(firstclass);
15     letterGroup.addService(firstclass);
16
17     // get all letter services
18     var letterServices = letterGroup.getServices();
19     ...
20 }

```

getShipmentEntryForms()

Type	Object method
Function Declaration	string[] getShipmentEntryForms()
Description	Returns an array of custom shipment entry form names added to a ServiceGroup with addShipmentEntryForm(entryForm). This method returns an empty array if no shipment entry forms are defined for the service group. For more information about adding shipment entry forms, see addShipmentEntryForm(entryForm). For more information about creating the custom entry forms for the Shipping Partner plug-in implementation, see Step 2. Customize Shipping Partner Entry Forms.
Returns	string[]
Input Parameters	None.
Parent object	ServiceGroup

Example

```

1  function getServiceGroups(input, output)
2  {
3      ...
4      // add package form name to service group
5      letterGroup.addShipmentEntryForm('First Class Letter Shipment Details')
6      letterGroup.addShipmentEntryForm('First Class Letter Shipment Details - Alternate');
7      ...
8      // get the names
9      var myShipmentForms = letterGroup.getShipmentEntryForms();
10     for (var i = 0; i < myShipmentForms.length; i++) {
11         ...
12     }

```

setCustomDataDescriptor(customDataDescriptor)

Type	Object method
Function Declaration	void setCustomDataDescriptor(CustomDataDescriptor customDataDescriptor)
Declaration	

Description Associates a [CustomDataDescriptor](#) object to a [ServiceGroup](#) object in a Shipping Partner plug-in implementation. Use this method if you want to access custom data when you calculate shipping rates or process a shipment. You can only set one CustomDataDescriptor object per service group.

To use this method, complete the following steps:

1. Create the [CustomDataDescriptor](#) object with [newCustomDataDescriptor\(\)](#).
2. Add the field values that you want to pass as custom data with [addItemParameter\(parameter,sourceField\)](#), [addPackageParameter\(parameter,sourceField\)](#), and [addShipmentParameter\(parameter,sourceField\)](#).
3. Use this method to add the CustomDataDescriptor object to the ServiceGroup object

For more information about using custom data, see [Understanding Custom Data in a Shipping Partner Plug-in Implementation](#).

Returns void

Input Parameters customDataDescriptor {[CustomDataDescriptor](#)} — CustomDataDescriptor object created with [newCustomDataDescriptor\(\)](#)

Parent object [ServiceGroup](#)

Example

```

1 function getServiceGroups(input, output)
2 {
3   ...
4   var letterGroup = output.getObjectFactory().newServiceGroup('letter');
5   ...
6   var customDataDescriptor = output.getObjectFactory().newCustomDataDescriptor();
7   customDataDescriptor
8     .addShipmentParameter('txn_memo', 'memo')
9     .addPackageParameter('pkg_description', 'packagedetails.description')
10    .addItemParameter('itm_exclude', 'excludefromraterequest');
11
12   letterGroup.setCustomDataDescriptor(customDataDescriptor);
13   ...
14 }
```

setId(id)

Type Object method

Function Declaration void setId(string id)

Description Defines the unique string ID for a [ServiceGroup](#). Use this method if you want to change the change the ID of a service group after you create it with [newServiceGroup\(id\)](#).

Returns void

Input Parameters id {string} — Unique ID for a service group. Must be a legal value or the method returns an error.

Parent object [ServiceGroup](#)

Example

```

1 function getServiceGroups(input, output)
2 {
3   ...
4   // create the service group
```

```

5   var letterGroup = output.getObjectFactory().newServiceGroup('letter');
6   ...
7   // set the letterGroup id
8   letterGroup.setId('lettergroup');
9   ...
10  }

```

setName(name)

Type Object method

Function Declaration void setName(string name)

Description Defines the name property for a [ServiceGroup](#). Use this method to set the name of a service group after you create it with [newServiceGroup\(id\)](#). This name appears as a service group name on the Shipping Rate and Shipping Label subtabs of the New Shipping Item page.

Returns void

Input Parameters name {string} — Name for a service group. Must be a legal value or the method returns an error.

Parent object [ServiceGroup](#)

Example

```

1  dfunction getServiceGroups(input, output)
2  {
3      ...
4      // create a service group and set the name
5      var letterGroup = output.getObjectFactory().newServiceGroup('letter');
6      letterGroup.setName('Letter Services');
7      ...
8  }

```

setRegistrationType(registrationType)

Type Object method

Function Declaration void setRegistrationType(string registrationType)

Description Defines the registration type for a [ServiceGroup](#). You must create a service group with [newServiceGroup\(id\)](#) before you set the registration type.

The registration type determines whether the services for a service group require calculation of real-time shipping rates or use shipping rates defined by the plug-in implementation only. For real-time rates, the plug-in connects to an external shipping partner account to calculate the rates. Otherwise, the plug-in implementation calculates the rates based on a formula defined by the implementation. You can set one of the following registration types for a service group:

- MANDATORY. All services calculate real-time shipping rates. The shipping partner must be registered with an external shipping account in NetSuite.
- OPTIONAL. All services calculate either real-time or standard shipping rates, depending on the shipping partner registration status in NetSuite.
- NONE. All services use shipping rates defined by the plug-in implementation only.

If you set MANDATORY or OPTIONAL, add a registration entry form to the service group with [addRegistrationEntryForm\(entryForm\)](#).

For more information about creating the custom entry forms for the Shipping Partner plug-in implementation, see [Step 2. Customize Shipping Partner Entry Forms](#).

Returns	void
Input Parameters	registrationType {string} — Registration type as a string for all services defined for a service group. Must be a legal value or the method returns an error.
Parent object	ServiceGroup

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     // create the service group
5     var letterGroup = output.getObjectFactory().newServiceGroup('letter');
6     ...
7     // set registration type and add registration form
8     letterGroup.setRegistrationType('MANDATORY');
9     letterGroup.addRegistrationEntryForm('Letter Shipping Partner Registration');
10    ...
11 }

```

Service

Type	Object
Description	The Service object defines a classification for how an item is shipped for a service group. You can define multiple services for a particular service group. For example, for a specific service group, items can be shipped first class, second class, or express. The services defined in the plug-in implementation appear on the Shipping Item page in NetSuite when you create a new shipping item and select the associated service group. A service object contains the following properties: ■ Name. String that defines the name of the service. This name appears as a service name on the Shipping Rate and Shipping Label subtabs of the New Shipping Item page. ■ Id. String that defines a unique identifier for the service. Each service within a service group must have a unique id. You set the ID of the service when you create it with newService(id). To create services for a service group: 1. Create the ServiceGroup with newServiceGroup(id). 2. Use newService(id) to create each required service and set the name with setName(name). 3. Add each service object to the associated service group object with addService(service). 4. Repeat the above steps for each additional service.
Methods	■ getId() ■ getName() ■ setId(id) ■ setName(name)
Parent Object(s)	ServiceGroup
Child Object(s)	n/a

getId()

Type	Object method
-------------	---------------

Function Declaration	string getId()
Description	Returns the unique ID of a Service . You must use newService(id) to create the service before you call this method.
Returns	string
Input Parameters	None.
Parent object	Service

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     // create a service group
5     var letterGroup = output.getObjectFactory().newServiceGroup('letter');
6     ...
7     // create the service, set name, and add to service group
8     var firstClass = output.getObjectFactory().newService('firstclass');
9     firstClass.setName('1st Class Mail');
10    lettergroup.addService(firstClass);
11    ...
12    // get the service id
13    var myServices = letterGroup.getServices();
14    var serviceId;
15    for (var i = 0; i < myServices.length; i++)
16    {
17        serviceId = myServices[i].getId();
18        // use service id
19        ...
20    }
21 }

```

getName()

Type	Object method
Function Declaration	string getName()
Description	Returns the name of a Service . You must use newService(id) and setName(name) to create the service and set the name before you call this method.
Returns	string
Input Parameters	None.
Parent object	Service

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     // create a service group
5     var letterGroup = output.getObjectFactory().newServiceGroup('letter');
6     ...
7     // create the service, set name, and add to service group
8     var firstClass = output.getObjectFactory().newService('firstclass');
9     firstClass.setName('1st Class Mail');
10    lettergroup.addService(firstClass);
11    ...
12    // get the service id
13    myServices = letterGroup.getServices();

```

```

14     var serviceName;
15     for (var i = 0; i < myServices.length; i++)
16     {
17         serviceName = myServices[i].getName();
18         // use service name
19         ...
20     }
21 }

```

setId(id)

Type	Object method
Function Declaration	void setId(string id)
Description	Defines the unique string ID for a Service . Use this method if you want to change the ID of a service group after you create it with newService(id) .
Returns	void
Input Parameters	id {string} — Unique ID for a service. Must be a legal value or the method returns an error.
Parent object	Service

Example

```

1 function getServiceGroups(input, output)
2 {
3     ..
4     var firstClass = output.getObjectFactory().newService('firstclass');
5     firstClass.setName('1st Class Mail');
6     ...
7     firstClass.setId('1stclass');
8     ...
9 }

```

setName(name)

Type	Object method
Function Declaration	void setName(string name)
Description	Defines the name property for a Service . Use this method to set the name of a service after you create it with newService(id) . This name appears as a service name on the Shipping Rate and Shipping Label subtabs of the New Shipping Item page.
Returns	void
Input Parameters	name {string} — Name for a service. Must be a legal value or the method returns an error.
Parent object	Service

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     // create the service and set the name
5     var firstClass = output.getObjectFactory().newService('firstclass');
6     firstClass.setName('1st Class Mail');
7     ...

```

8 | }

CustomDataDescriptor

Type	Object
Description	Object that represents all field values to be passed as custom data to a Shipping Partner plug-in implementation for a specific service group. A CustomDataDescriptor object contains key-value pairs for any item, shipment, and package field in an item fulfillment or Web store order that you want to access in the plug-in implementation. For each key-value pair, the key is the parameter name defined in the plug-in implementation. The value is the actual field value passed to the plug-in implementation from the transaction. For a shipment, the field name can be from either an item fulfillment field, like memo, or from the Shipment Details subrecord, like shipmentsdetails.description. Similarly, for a package, the field can be from the Packages sublist, like description, or from the Package Details subrecord, like package.description. For an item, the field can be the item record or item sublist on the transaction. To use this object, complete the following steps: 1. Identify the item, shipment, or package field names that you want to pass to the plug-in implementation. 2. Create the object with newCustomDataDescriptor(). 3. Add the parameters with addItemParameter(parameter,sourceField), addPackageParameter(parameter,sourceField), or addShipmentParameter(parameter,sourceField). For more information about using custom data, see Understanding Custom Data in a Shipping Partner Plug-in Implementation.
Methods	■ addItemParameter(parameter,sourceField) ■ addPackageParameter(parameter,sourceField) ■ addShipmentParameter(parameter,sourceField) ■ getItemParameters() ■ getPackageParameters() ■ getShipmentParameters()
Parent Object(s)	ServiceGroup
Child Object(s)	n/a

addItemParameter(parameter,sourceField)

Function Declaration	CustomDataDescriptor addItemParameter(string parameter,string sourceField)
Type	Object method
Description	Adds an item field from an item fulfillment or Web store order to the CustomDataDescriptor object for a service group. You can use any field available on an item record or the item sublist on the transaction. NetSuite passes the parameter name and field value as a key-value pair to the plug-in implementation. You can retrieve the field value for all items as custom data. You can access this custom data with one of the following methods: ■ getCustomData(key)

- [getCustomData\(key,def\)](#)

Returns [CustomDataDescriptor](#)

Input Parameters

- `parameter {string}` — Unique key value for the field.
- `sourceField {string}` — NetSuite internal ID for the field. This can be a custom field or a standard NetSuite field.

Parent object [CustomDataDescriptor](#)

Example

```

1 function getServiceGroups(input, output)
2 {
3   ...
4   var letterGroup = output.getObjectFactory().newServiceGroup('letter');
5   ...
6   var customDataDescriptor = output.getObjectFactory().newCustomDataDescriptor();
7   customDataDescriptor.addItemParameter('itm_exclude', 'excludefromraterequest');
8   ...
9   letterGroup.setCustomDataDescriptor(customDataDescriptor);
10  ...
11 }
```

addPackageParameter(parameter,sourceField)

Function Declaration `CustomDataDescriptor addPackageParameter(string parameter,string sourceField)`

Type Object method

Description Adds a package field from an item fulfillment or Web store order to the [CustomDataDescriptor](#) object for a service group. You can use any field from the Packages sublist or any field on the Package Details subrecord. NetSuite passes the parameter name and field value as a key-value pair to the plug-in implementation.

You can retrieve the field value for package custom data when you calculate rates or process the shipment. If the field is a custom field, you can also set the field value to be passed back to NetSuite. For more information, see the methods for the [Package](#), [CustomDataContainer](#), and [CustomData](#) objects.

Returns [CustomDataDescriptor](#)

Input Parameters

- `parameter {string}` — Unique key value for the field.
- `sourceField {string}` — NetSuite internal ID for the field. This can be a custom field or a standard NetSuite field.

Parent object [CustomDataDescriptor](#)

Example

```

1 function getServiceGroups(input, output)
2 {
3   ...
4   var letterGroup = output.getObjectFactory().newServiceGroup('letter');
5   ...
6   var customDataDescriptor = output.getObjectFactory().newCustomDataDescriptor();
7   customDataDescriptor.addPackageParameter('pkg_description', 'packagedetails.description');
8   ...
9   letterGroup.setCustomDataDescriptor(customDataDescriptor);
10  ...
11 }
```


addShipmentParameter(parameter,sourceField)

Function Declaration	CustomDataDescriptor addShipmentParameter(string parameter,string sourceField)
Type	Object method
Description	Adds a shipment field from an item fulfillment to the CustomDataDescriptor object for a service group. The field can be any item fulfillment or Web store order field or any field on the Shipment Details subrecord. NetSuite passes the parameter name and field value as a key-value pair to the plug-in implementation. You can retrieve the field value for shipment custom data when you calculate rates or process the shipment. If the field is a custom field, you can also set the field value to be passed back to NetSuite. For more information, see the methods for the Package, CustomDataContainer, and CustomData objects.
Returns	CustomDataDescriptor
Input Parameters	■ parameter {string} — Unique key value for the field. ■ sourceField {string} — NetSuite internal ID for the field. This can be a custom field or a standard NetSuite field.
Parent object	CustomDataDescriptor

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     var letterGroup = output.getObjectFactory().newServiceGroup('letter');
5     ...
6     var customDataDescriptor = output.getObjectFactory().newCustomDataDescriptor();
7     customDataDescriptor.addShipmentParameter('txn_memo', 'memo');
8     ...
9     letterGroup.setCustomDataDescriptor(customDataDescriptor);
10    ...
11 }

```

getItemParameters()

Function Declaration	Map<string,string> getItemParameters()
Type	Object method
Description	Returns a Javascript map that contains key-value pairs for all item fields added to a CustomDataDescriptor object by addItemParameter(parameter,sourceField). Use this method to retrieve all item fields added for a specific service group. This method returns the map data in the following format:

```

1 {
2   "parameter1" : "itemFieldName1"
3   "parameter2" : "itemFieldName2"
4   ...
5   "parameterN" : "itemFieldNameN"
6 }

```

This method returns an empty map object if no item fields have been added to the CustomDataDescriptor object.

Returns map object

Input Parameters None.

Parent object [CustomDataDescriptor](#)

Example

```
1 function getServiceGroups(input, output)
2 {
3     ...
4     // get item parameters for service group 'lettergroup'
5     var itemParams = letterGroup.getCustomDataDescriptor().getItemParameters();
6     for (var key in itemParams) {
7         thisParam = itemParams[key];
8         ...
9     }
10    ...
11 }
```

getPackageParameters()

Function Declaration Map<string,string> getPackageParameters()

Type Object method

Description Returns a Javascript map that contains key-value pairs for all package fields added to a [CustomDataDescriptor](#) object by [addPackageParameter\(parameter,sourceField\)](#). Use this method to retrieve all package fields added for a specific service group.

This method returns the map data in the following format:

```
1 {
2   "parameter1" : "packageFieldName1"
3   "parameter2" : "packageFieldName2"
4   ...
5   "parameterN" : "packageFieldNameN"
6 }
```

This method returns an empty map object if no package fields have been added to the CustomDataDescriptor object.

Returns map object

Input Parameters None.

Parent object [CustomDataDescriptor](#)

Example

```
1 function getServiceGroups(input, output)
2 {
3     ...
4     // get package parameters for service group 'lettergroup'
5     var pkgParams = letterGroup.getCustomDataDescriptor().getPackageParameters();
```

```

6      for (var key in pkgParams) {
7          thisParam = pkgParams[key];
8          ...
9      }
10     ...
11 }

```

getShipmentParameters()

Function Declaration Map<string,string> getShipmentParameters()

Type Object method

Description Returns a Javascript map that contains key-value pairs for all shipment fields added to a [CustomDataDescriptor](#) object by [addShipmentParameter\(parameter, sourceField\)](#). Use this method to retrieve all shipment fields added for a specific service group.

This method returns the map data in the following format:

```

1  {
2    "parameter1" : "shipmentFieldName1"
3    "parameter2" : "shipmentFieldName2"
4    ...
5    "parameterN" : "shipmentFieldNameN"
6  }

```

This method returns an empty map object if no shipment fields have been added to the CustomDataDescriptor object.

Returns map object

Input Parameters None.

Parent object [CustomDataDescriptor](#)

Example

```

1  function getServiceGroups(input, output)
2  {
3      ...
4      // get shipment parameters for service group 'lettergroup'
5      var shipParams = letterGroup.getCustomDataDescriptor().getShipmentParameters();
6      for (var key in shipParams) {
7          thisParam = shipParams[key];
8          ...
9      }
10     ...
11 }

```

displayRegistration(DisplayRegistrationInput input, DisplayRegistrationOutput output)

Function | Method Declaration void displayRegistration(DisplayRegistrationInput input, DisplayRegistrationOutput output)

Type	Interface function
Description	Defines the shipping partner registration credentials that appear on the Shipping Partner Registration page. This function defines the external shipping account credentials that are required to connect to the external shipping partner account to calculate real-time rates and generate shipping documents. This function is called by NetSuite when a user opens a shipping partner registration form. The credentials defined with this function are placed on the shipping partner registration page and stored by the processRegistration(ProcessRegistrationInput input, ProcessRegistrationOutput output) interface function when a user submits the registration form.
Returns	void
Parameters	■ DisplayRegistrationInput ■ DisplayRegistrationOutput

DisplayRegistrationInput

Type	Object
Description	Input object for the displayRegistration(DisplayRegistrationInput input, DisplayRegistrationOutput output) interface function.
Methods	■ getCredentialsRecordKey() ■ getServiceGroupId() ■ getLocale()
Child Objects	n/a

getCredentialsRecordKey()

Function Declaration	
Type	Object method
Description	Returns the key of the record in NetSuite that stores the user credentials for a shipping partner registration. Use this key to retrieve the user credentials for the external shipping partner account associated with a specific service group. Use these values to populate the fields on the shipping partner registration form. If the user credentials have not yet been added to NetSuite by the addCredentialField(name, label, domains, allowedUserScriptId, value) method, this method returns null. Depending on your requirements, you can use this method in conjunction with getServiceGroupId() to display the desired user credentials, based on the service group selected on the Shipping Partner Registration page. For more information about the user credentials and registering an external shipping partner account, see displayRegistration(DisplayRegistrationInput input, DisplayRegistrationOutput output) and Register a Shipping Partner.
Returns	string NULL
Input Parameters	None.
Parent object	DisplayRegistrationInput

Example

```

1 function displayRegistration(input, output)
2 {
3     ...
4     var id = input.getCredentialsRecordKey();
5
6     if (id == null)
7     {
8         output.addCredentialField('username', 'User Name', DOMAIN, scriptId, null);
9         output.addCredentialField('password', 'Password', DOMAIN, scriptId, null);
10    }
11    else
12    {
13        var record = nlapiLoadRecord(RECORD_NAME, id);
14        output.addCredentialField('username', 'User Name', DOMAIN, scriptId, record.getFieldValue(LOGIN_FIELD));
15        output.addCredentialField('password', 'Password', DOMAIN, scriptId, record.getFieldValue(PWD_FIELD));
16    }
17    ...
18 }

```

getServiceGroupId()

Function Declaration string getServiceGroupId()

Type Object method

Description Returns service group ID of the currently selected service group on the **Shipping Partner Registration** page. Use this function, for example, to customize the shipping partner registration page that displays for different service groups.

Returns string

Input Parameters None.

Parent object [DisplayRegistrationInput](#)

Example

```

1 function displayRegistration(input, output)
2 {
3     ...
4     var serviceGroupId = input.getServiceGroupId();
5
6     if (serviceGroupId == 'letter')
7     {
8         output.addRegistrationEntryForm('Letter Shipping Partner Registration');
9     }
10    if (serviceGroupId == 'parcel')
11    {
12        output.addRegistrationEntryForm('Parcel Shipping Partner Registration');
13    }
14    ...
15 }

```

DisplayRegistrationOutput

Type Interface output object

Description The DisplayRegistrationOutput object contains the method to add user credential fields to a shipping partner registration form. DisplayRegistrationOutput is the output object for the

`displayRegistration(DisplayRegistrationInput input, DisplayRegistrationOutput output)` interface function.

- Methods**
- `addCredentialField(name, label, domains, allowedUserScriptId, value)`
 - `getNotificationsContainer()`
 - `getObjectFactory()`

- Child Objects**
- `NotificationsContainer`

addCredentialField(name, label, domains, allowedUserScriptId, value)

Function Declaration `void addCredentialField(string name, string label, string[] domains, string allowedUserScriptId, string value)`

Type Object method

Description Adds a user credential field to the shipping partner registration form. After the user saves the registration form, these fields are stored by NetSuite as a user credentials record and used to validate the user credentials with the external shipping partner account.

Returns void

- Input Parameters**
- `name {string}` — Name for the credential field.
 - `label {string}` — Label for the credential field.
 - `domain(s) {string | string[]}` — Domain or array of domains to which the credentials can be sent for verification. If only one domain, can be a single string.
 - `allowedUserScriptID {string}` — Script ID that can access this field. Use the SuiteScript `nlapiGetContext()` method to get the script ID of the plug-in script.
 - `value {string}` — Value of the credential field. If the user credential record already exists in NetSuite, use the SuiteScript `nlapiLoadRecord(type, id, initializeValues)` method to access the record values and populate the field value.

Throws `java.sql.SQLException, oracle.net.jdbc.nl.NLException`

Parent object `DisplayRegistrationOutput`

Input Parameters

Example

```

1 function displayRegistration(input, output)
2 {
3     var scriptId = nlapiGetContext().getScriptId();
4
5     var id = input.getCredentialsRecordKey();
6     if (id == null)
7     {
8         output.addCredentialField('username', 'User Name', DOMAIN, scriptId, null);
9         output.addCredentialField('password', 'Password', DOMAIN, scriptId, null);
10    }
11    else
12    {
13        var record = nlapiLoadRecord(RECORD_NAME, id);
14        output.addCredentialField('username', 'User Name', DOMAIN, scriptId, record.getFieldValue(LOGIN_FIELD));
15        output.addCredentialField('password', 'Password', DOMAIN, scriptId, record.getFieldValue(PWD_FIELD));
16    }
17 }
```

processRegistration(ProcessRegistrationInput input, ProcessRegistrationOutput output)

Function Declaration	void processRegistration(ProcessRegistrationInput input, ProcessRegistrationOutput output)
Type	Interface function
Description	Defines the plug-in implementation functionality when a user submits a shipping partner registration form. This function stores the credential fields for an external shipping partner account entered on the shipping partner registration entry form.
Returns	void
Parameters	■ ProcessRegistrationInput ■ ProcessRegistrationOutput

ProcessRegistrationInput

Type	Interface input object
Description	The DisplayRegistrationInput object contains the objects and properties that are provided by NetSuite as input to the processRegistration(ProcessRegistrationInput input, ProcessRegistrationOutput output) interface function.
Methods	■ getCredentialFieldValue(name) ■ getCredentialsRecordKey() ■ getServiceGroupId() ■ getLocale()
Child Objects	n/a

getCredentialFieldValue(name)

Function Declaration	string getCredentialFieldValue(string name)
Type	Object method
Description	Returns the string value of the user credential field name entered in the Shipping Partner Registration form. Use this value to populate the record for the shipping partner user credentials stored by NetSuite. This method throws <code>java.lang.Exception</code> if the form field is empty. For more information, see the help topic Register a Shipping Partner.
Returns	string
Input Parameters	name {string} — Registration form field name.
Throws	java.lang.Exception
Parent object	ProcessRegistrationInput

Example

```

1 function processRegistration(input, output)
2 {
3     ...
4     // get the username and password field values
5     var username = input.getCredentialFieldValue('username');
6     var password = input.getCredentialFieldValue('password');
7     ...
8 }

```

getCredentialsRecordKey()

Function Declaration

Type Object method

Description Returns the key of the record in NetSuite that stores the user credentials for a shipping partner registration. Use this key to update the user credentials record in NetSuite with the values submitted in the shipping partner registration form. If the user credentials have not yet been added to NetSuite, this method returns null.

Depending on your requirements, you can use this method in conjunction with [getServiceGroupId\(\)](#) to retrieve the desired user credential record, based on the service group selected on the **Shipping Partner Registration** page.

For more information about the user credentials and registering an external shipping partner account, see [processRegistration\(ProcessRegistrationInput input, ProcessRegistrationOutput output\)](#) and [Register a Shipping Partner](#).

Returns string | NULL

Input Parameters None.

Parent object [ProcessRegistrationInput](#)

Example

```

1 function processRegistration(input, output)
2 {
3     var key = input.getCredentialsRecordKey();
4     var username = input.getCredentialFieldValue('username');
5     var password = input.getCredentialFieldValue('password');
6
7     if (key == null)
8     {
9         // create the credential record instance
10    }
11    else
12    {
13        // load and update record
14    }
15    // set the credentials record key for the current service group
16    output.setCredentialsRecordKey(key);
17 }

```

getServiceGroupId()

Function Declaration

Type Object method

Description	Returns service group ID of the currently selected service group on the Shipping Partner Registration page. Use this function, for example, to customize the registration process depending on the service group.
Returns	string
Input Parameters	None.
Parent object	ProcessRegistrationInput

Example

```

1 function processRegistration(input, output)
2 {
3     ...
4     // code sample line
5     ...
6 }

```

ProcessRegistrationOutput

Type	Interface output object
Description	The ProcessRegistrationOutput object contains methods to access the user credential record for the service group and the user credential fields on the current Shipping Partner Registration page. ProcessRegistrationOutput is the output object for the processRegistration(ProcessRegistrationInput input, ProcessRegistrationOutput output) interface function.
Methods	■ getCredentialsRecordKey() ■ setCredentialsRecordKey(key) ■ getNotificationsContainer() ■ getObjectFactory()
Child Objects	■ NotificationsContainer

getCredentialsRecordKey()

Function Declaration	string getCredentialsRecordKey()
Type	Object method
Description	Returns the key of the record in NetSuite that stores the user credentials for a shipping partner registration. Use this key to update the user credentials record in NetSuite with the values submitted in the shipping partner registration form. If the user credentials have not yet been added to NetSuite, this method returns null.
Returns	string NULL
Input Parameters	None.
Parent object	ProcessRegistrationOutput

Example

```

1 function processRegistration(input, output)
2 {

```

```

3      ...
4      // code sample line
5      ...
6  }
```

setCredentialsRecordKey(key)

Function Declaration	void setCredentialsRecordKey(string key)
Type	Object method
Description	Sets the record key for the user credentials record in NetSuite that is associated with the service group and user credential fields on the Shipping Partner Registration page.
Returns	void
Input Parameters	key {string} — String for the user credentials record key.
Parent object	ProcessRegistrationOutput

calculateRates(CalculateRatesInput input, CalculateRatesOutput output)

Function Declaration	void calculateRates(CalculateRatesInput input, CalculateRatesOutput output)
Type	Interface function
Description	Define the plug-in implementation functionality when a user selects a shipping method for the shipping partner and calculates shipping rates on a sales order, item fulfillment, or Web store order. The interface input object CalculateRatesInput provides the sales order, item fulfillment, and Web store order properties, including all items, packages, address and contact information, and shipping method properties. Use these properties to define shipping partner-specific logic to calculate shipping rates. Use the interface output object CalculateRatesOutput to create the shipping rates and pass the rates back to NetSuite.
Returns	void
Parameters	■ CalculateRatesInput ■ CalculateRatesOutput

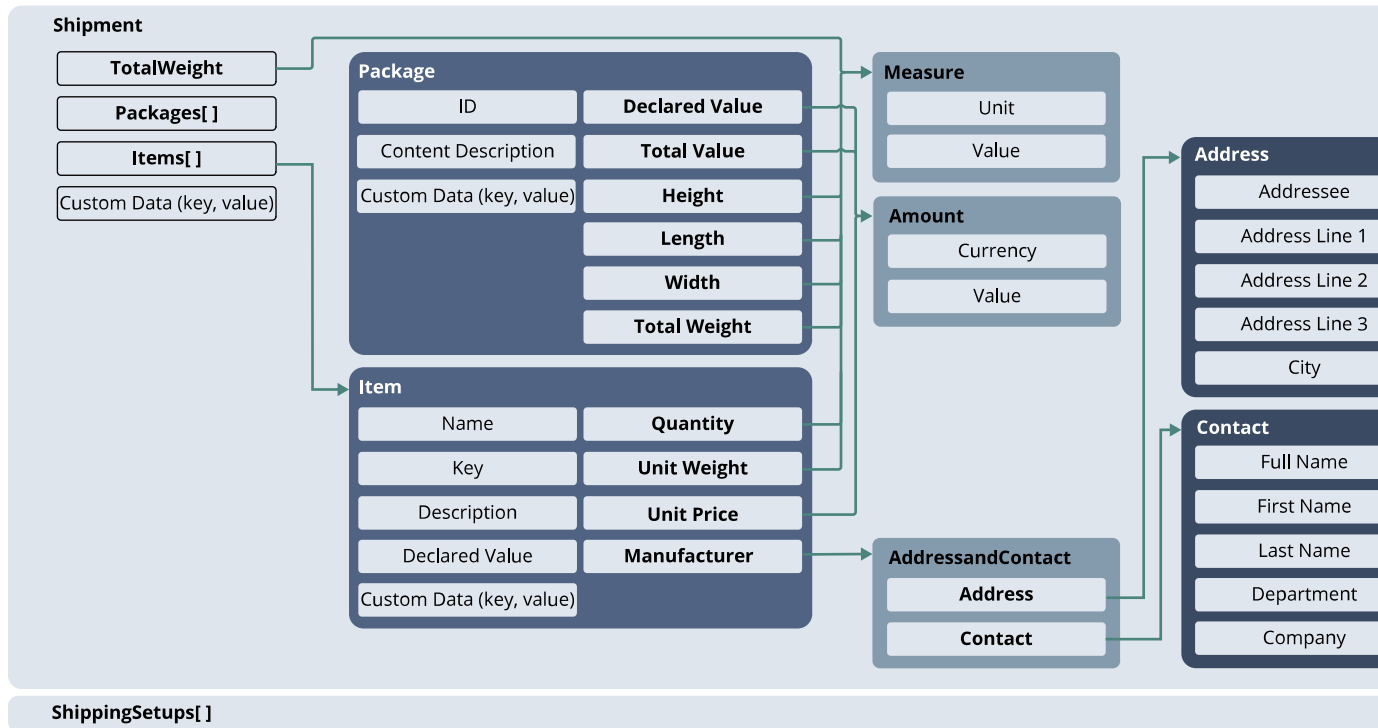
CalculateRatesInput

Type	Interface input object
Description	The CalculateRatesInput object is the input object for the calculateRates(CalculateRatesInput input, CalculateRatesOutput output) interface function.
Methods	■ getShippingDefinition() ■ getLocale()
Child Objects	ShippingDefinition

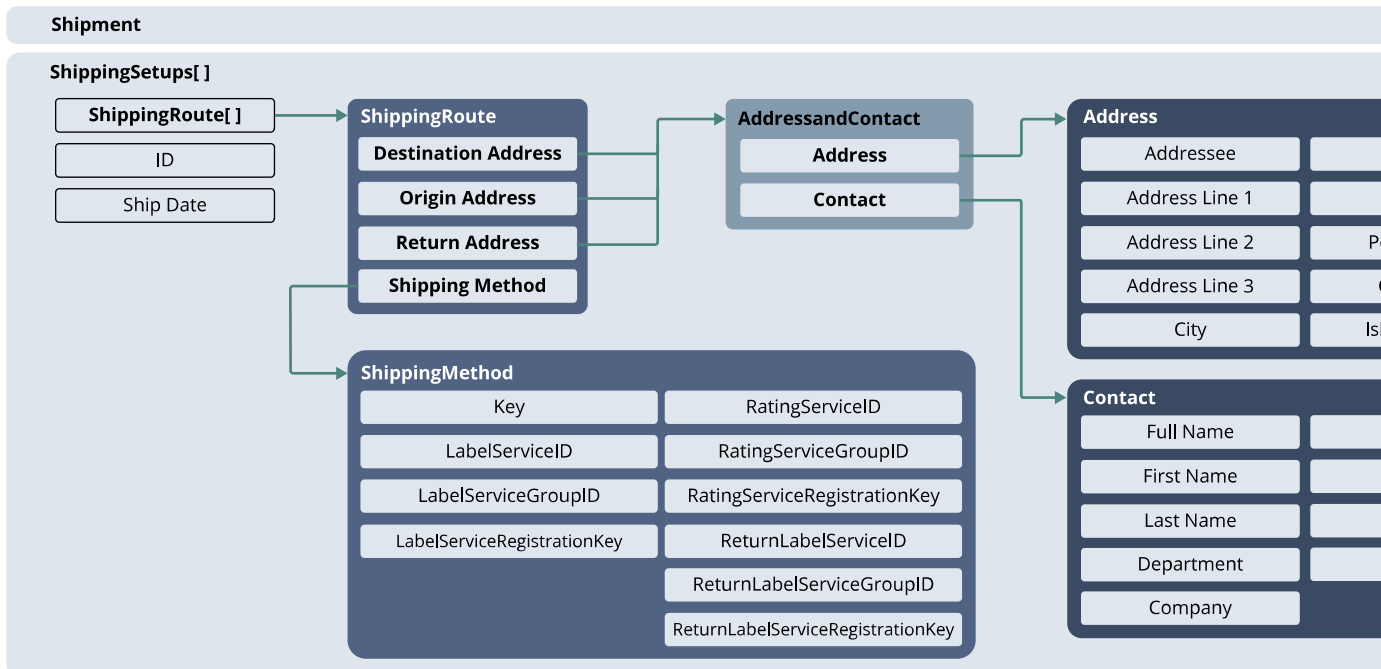
CalculateRatesInput Object Model

The following two figures show the CalculateRatesInput object model:

ShippingDefinition



ShippingDefinition



getShippingDefinition()

Function Declaration	ShippingDefinition getShippingDefinition()
Type	Object method
Description	Returns a ShippingDefinition object that contains the shipping details for a sales order or item fulfillment transaction. Use this object to access the transaction details. You can also use this method with getShipment() and getShippingSetups() , depending on your requirements.



Note: This method is available from both the [CalculateRatesInput](#) and [ProcessRegistrationInput](#) interface input objects.

Returns	ShippingDefinition
Input Parameters	None.
Parent object	CalculateRatesInput , ProcessRegistrationInput

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     // get shipping definition
5     var allDetails = input.getShippingDefinition();
6     ...
7     // access Shipment and ShippingSetup objects individually
8     var shippingDetails = input.getShippingDefinition().getShipment();
9     var shippingMethods = input.getShippingDefinition().getShippingSetups();
10    ...
11 }

```

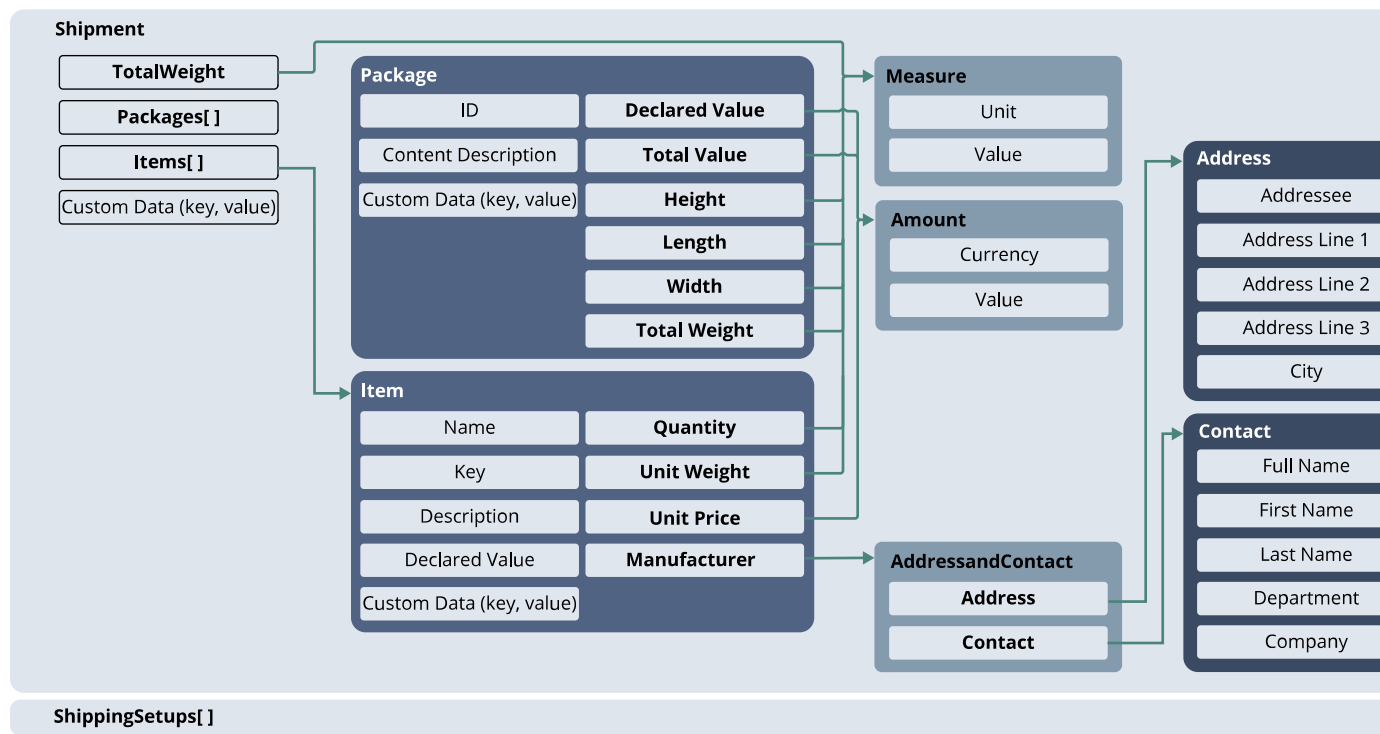
ShippingDefinition

Type	Object
Description	The ShippingDefinition object contains the shipping data for a sales order or item fulfillment transaction. Use this object to access the items and packages for the transaction as well as the selected shipping methods for the transaction. This object contains the following data: ■ Sales order data. Data from the Items and Shipping subtabs of a sales order. ■ Item fulfillment data. Data from the Items, Shipping, and Packages subtabs of an item fulfillment, as well as the data from the Shipment Details record form.
Methods	■ getShipment() ■ getShippingSetups()
Parent Object(s)	CalculateRatesInput , ProcessShipmentInput
Child Object(s)	Shipment , ShippingSetup

ShippingDefinition Object Model

The following figure shows the ShippingDefinition object model:

ShippingDefinition



getShipment()

Function Declaration Shipment getShipment()

Type Object method

Description Returns a [Shipment](#) object. Use this method to access the item and package data for a sales order or item fulfillment and the total weight of a shipment. This method returns an empty value if there are no items or packages on the respective subtabs of the transaction.

Returns [Shipment](#)

Input Parameters None.

Parent object [CalculateRatesInput](#)

Example

```
1 function calculateRates(input, output)
2 {
3     ...
4     // get item and package data
5     var shippingDetails = input.getShippingDefinition().getShipment();
6     var allItems = shippingDetails.getShipment().getItems();
7     var allPackages = shippingDetails.getShipment().getPackages();
8     ...
9 }
```

getShippingSetups()

Function Declaration ShippingSetup[] getShippingSetups()

Type	Object method
Description	Returns an array of ShippingSetup objects. A single sales order or item fulfillment transaction can contain details for multiple shipping methods if the Multiple Shipping Routes feature is used.
Returns	ShippingSetup[]
Input Parameters	None.
Parent object	CalculateRatesInput

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     // access Shipment and ShippingSetup objects individually
5     var shippingDetails = input.getShippingDefinition().getShipment();
6     var shippingMethods = input.getShippingDefinition().getShippingSetups();
7     ...
8 }

```

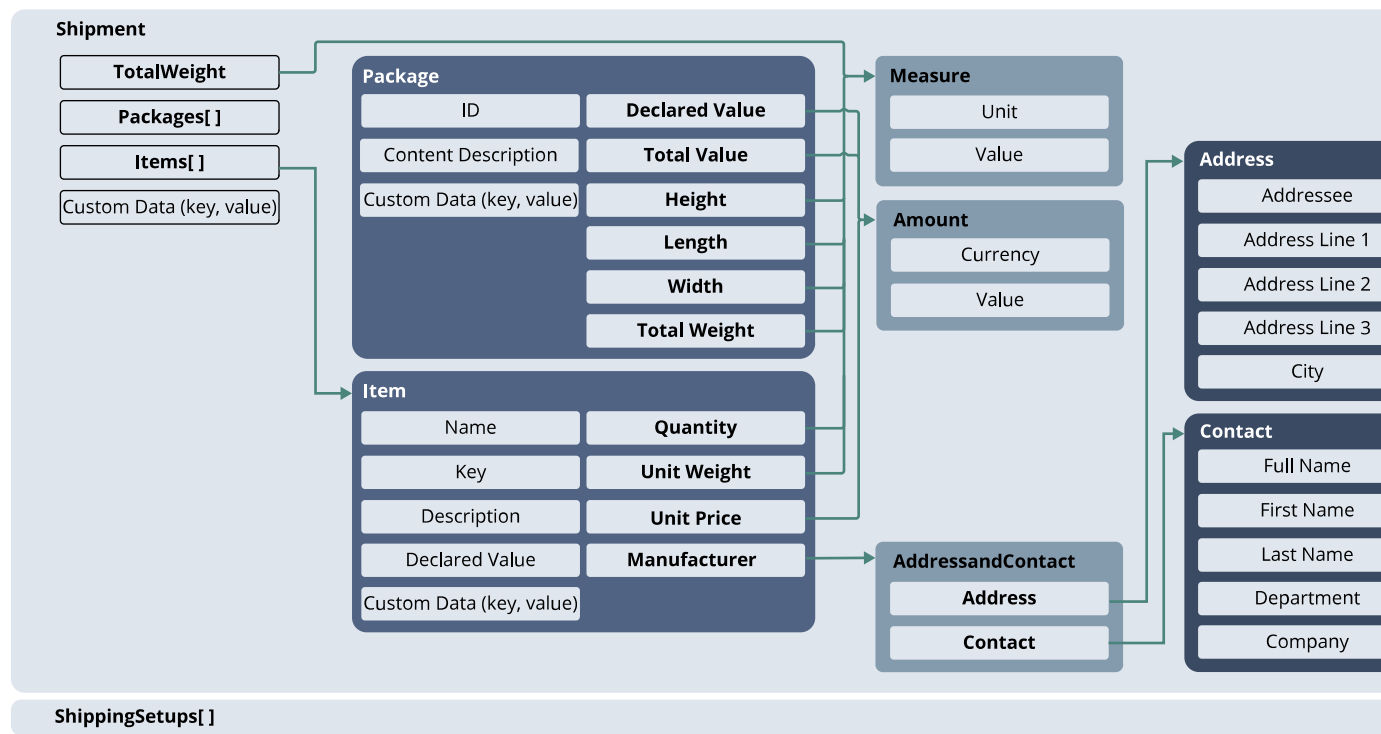
Shipment

Type	Object
Description	The Shipment object contains the data for a single sales order or item fulfillment transaction. This data includes all items for a sales order transaction, and all items and packages from the Items and Packages subtabs for a an item fulfillment transaction. It also includes the calculated total weight of the shipment as a Measure object and any custom data added with addShipmentParameter(parameter,sourceField). Use the data from the items and packages when calculating shipping rates or processing shipments. Only one Shipment object is passed per transaction to the interface functions.
Methods	■ getCustomData(key,def) ■ getShipment() ■ getItems() ■ getPackages() ■ getTotalWeight()
Parent Object(s)	ShippingDefinition
Child Object(s)	■ Item ■ Package ■ Measure

Shipment Object Model

The following figure shows the Shipment object model:

ShippingDefinition



getCustomData(key)

Function Declaration string getCustomData(string key)

Type Object method

Description Returns the custom data for a shipment field key. Use this method to retrieve a value for a shipment field added with [addShipmentParameter\(parameter,sourceField\)](#). This method returns null if the key has not been added as a parameter or if the field on the transaction has no value.

For example, you add the **Memo** field as custom data in the [getServiceGroups\(GetServiceGroupsInput input, GetServiceGroupsOutput output\)](#) interface function:

```
1 | customDataDescriptor.addShipmentParameter('txn_memo', 'memo');
```

You can access this field value when calculating rates or processing a shipment:

```
1 | var memoField = shipment.getCustomData("txn_memo");
```

Returns string

Input Parameters key {string} — Key value for the field specified as the parameter input argument in [addShipmentParameter\(parameter,sourceField\)](#).

Parent object [Shipment](#)

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     var shippingDef = input.getShippingDefinition();
5     var shipment = shippingDef.getShipment();
6
7     var memoField = shipment.getCustomData("txn_memo");
8     ...
9 }

```

getCustomData(key,def)

Function Declaration	string getCustomData(string key, string def)
Type	Object method
Description	Returns the custom data for a shipment field key or returns a default value. Use this method to retrieve a value for a shipment field added with addShipmentParameter(parameter,sourceField) or, if the key does not exist, use a default value. You can use this method or use getCustomData(key) to retrieve the field value for shipment custom data.
Returns	string
Input Parameters	■ key {string} — Key for the shipment field specified as the parameter input argument in addShipmentParameter(parameter,sourceField). ■ def {string} — Field value to return if the key does not exist.
Parent object	Shipment

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     var shippingDef = input.getShippingDefinition();
5     var shipment = shippingDef.getShipment();
6
7     var defaultMemo = "Rate calculated by a Shipping Partner plug-in."
8
9     var memoField = shipment.getCustomData("txn_memo",defaultMemo);
10    ...
11 }

```

getItems()

Function Declaration	Item[] getItems()
Type	Object method
Description	Returns an array of Item objects. The Item object includes all items from the Items sublist of a sales order or item fulfillment transaction.
Returns	Item[]
Input Parameters	None.
Parent object	Shipment

Example

```

1 dfunction calculateRates(input, output)
2 {
3     ...
4     // get item and package data
5     var shippingDetails = input.getShippingDefinition().getShipment();
6     var allItems = shippingDetails.getShipment().getItems();
7     var allPackages = shippingDetails.getShipment().getPackages();
8     ...
9     processAllItems(allItems);
10    ...
11 }

```

getPackages()

Function Declaration	Package[] getPackages()
Type	Object method
Description	Returns an array of Package objects. The Package objects include the data from the Packages subtab of an item fulfillment transaction.
Returns	Package []
Input Parameters	None.
Parent object	Shipment

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     // get item and package data
5     var shippingDetails = input.getShippingDefinition().getShipment();
6     var allItems = shippingDetails.getShipment().getItems();
7     var allPackages = shippingDetails.getShipment().getPackages();
8     ...
9     processAllPackages(allPackages);
10    ...
11 }

```

getTotalWeight()

Function Declaration	
Type	Object method
Description	Returns a Measure objects that represents the total weight of the shipment.
Returns	Measure
Input Parameters	None.
Parent object	Shipment

Example

```

1 function calculateRates(input, output)
2 {
3     ...

```

```

4 // get item and package data
5 var shippingDetails = input.getShippingDefinition().getShipment();
6 var totalWeight = shippingDetails.getShipment().getTotalWeight();
7 ...
8 }

```

Item

Type Object

Description An Item object contains the data for a single item line on the Items subtab of a sales order or item fulfillment transaction. Use the object methods to access individual properties of the item, including custom data. The data you access with the object methods is read-only. You cannot use the plug-in logic to modify the item properties.



Note: If an item in a sales order has the **Exclude Item From Rate Request** field checked, the item is not sent to the plug-in implementation.

Parent Objects [Shipment](#)

Child Objects [Amount](#), [AddressandContact](#), [Measure](#)

The following table lists the Item object methods and the location of the values on the transaction that they return:

Method	Return Type	Description
getCustomData(key)	string	See getCustomData(key) .
getCustomData(key,def)	string	See getCustomData(key,def) .
getDeclaredValue()	Amount	On a sales order, returns the value of Amount , located on the Item sublist. On an item fulfillment, returns the value of the <code>itemfxamount</code> hidden field, which equals the individual item cost multiplied by the quantity (unit price x quantity).
getDescription()	string	Items subtab
getKey()	string	Returns the internal NetSuite ID for the item. Use this key with nlapiLoadRecord(type, id, initializeValues) to load an item record.
getManufacturer()	AddressandContact	Item record
getName()	string	Item subtab
getQuantity()	Measure	Item subtab
getUnitPrice()	Amount	Item record
getUnitWeight()	Measure	Item record

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     // get item and package data
5     var shippingDetails = input.getShippingDefinition().getShipment();
6     var allItems = shippingDetails.getShipment().getItems();
7     var allPackages = shippingDetails.getShipment().getPackages();

```

```

8      ...
9      for (var i = 0; i < allItems.length; i++)
10     {
11         total += processOneItem(allItems[i]);
12     }
13     ...
14 }

```

getCustomData(key)

Function Declaration string getCustomData(string key)

Type Object method

Description Returns the custom data for an item key value. Use this method to retrieve a value for an item field added with [addItemParameter\(parameter,sourceField\)](#). Returns null if the key has not been added as a parameter or if the field has no value on the transaction.

For example, you add the **Exclude Item From Rate Request** field as custom data in the [getServiceGroups\(GetServiceGroupsInput input, GetServiceGroupsOutput output\)](#) interface function:

```
1 | customDataDescriptor.addItemParameter('itm_exclude', 'excludefromraterequest');
```

You can access this field value when calculating rates or processing a shipment:

```
1 | var excludeField = item.getCustomData("itm_exclude");
```

Returns string

Input Parameters key {string} — Key value for the field specified as the parameter input argument with [addItemParameter\(parameter,sourceField\)](#).

Parent object [Item](#)

Example

```

1 | function calculateRates(input, output)
2 | {
3 |     ...
4 |     var shippingDef = input.getShippingDefinition();
5 |     var items = shippingDef.getItems();
6 |
7 |     for (var i = 0; i < allItems.length; i++)
8 |     {
9 |         // if item is not excluded from rates request
10 |         if (!items[i].getCustomData('itm_exclude')) {
11 |             // go ahead and process the item
12 |             ...
13 |         } else {
14 |             // do something else
15 |         }
16 |         ...
17 |     }

```

getCustomData(key,def)

Function Declaration string getCustomData(string key, string def)

Type Object method

Description	Returns the custom data for an item key value or returns a default value. Use this method to retrieve a value for an item field added with addItemParameter(parameter,sourceField) or, if the key does not exist, use a default value. You can use this method or use getCustomData(key) to retrieve the field value for item custom data.
Returns	string
Input Parameters	■ key {string} — Key for the item field specified as the parameter input argument with addItemParameter(parameter,sourceField). ■ def {string} — Field value to return if the key does not exist.
Parent object	Item

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     var shippingDef = input.getShippingDefinition();
5     var items = shippingDef.getItems();
6
7     var defItemDesc = "Item has no description";
8
9     for (var i = 0; i < allItems.length; i++)
10    {
11        var thisItemDesc = items[i].getCustomData('itm_desc',defItemDesc);
12        ...
13    }
14    ...
15 }
```

Package

Type	Object
Description	A Package object contains the data for a single package line on the Packages subtab of an item fulfillment transaction. Use the object methods to access individual properties of the package, including custom data.
Parent Objects	Shipment
Child Objects	Amount , Measure

The following table lists the Package object methods:

Method	Return Type	Return Type Description
getCustomData(key)	string	See getCustomData(key) .
getCustomData(key,def)	string	See getCustomData(key,def) .
getContentDescription()	string	Packages sublist
getDeclaredValue()	Amount	Package Details subrecord
getHeight()	Measure	Package Details subrecord
getId()	string	Returns the ID of the package record. Use to associate ShipmentDocument or CustomData objects with a specific package.

Method	Return Type	Return Type Description
getLength()	Measure	Package Details subrecord
getTotalValue()	Amount	Packages sublist
getTotalWeight()	Measure	Packages sublist
getWidth()	Measure	Package Details subrecord

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     // get item and package data
5     var shippingDetails = input.getShippingDefinition().getShipment();
6     var allItems = shippingDetails.getShipment().getItems();
7     var allPackages = shippingDetails.getShipment().getPackages();
8     ...
9     var total = 0;
10    for (var i = 0; i < allPackages.length; i++)
11    {
12        total += processOnePackage(allPackages[i]);
13    }
14    ...
15 }

```

getCustomData(key)

Function Declaration string getCustomData(string key)

Type Object method

Description Returns the custom data for a package key value. Use this method to retrieve a value for a package field added with [addPackageParameter\(parameter,sourceField\)](#). Returns null if the key has not been added as a parameter or if the field has no value on the transaction.

For example, you add the **Description** field from the Package Details subrecord as custom data in the interface function:

```

1 | customDataDescriptor.addItemParameter('pkg_description', 'package.description');

```

You can access this field value when calculating rates or processing a shipment:

```

1 | var descField = package.getCustomData('pkg_description');

```

You can use this method or use [getCustomData\(key,def\)](#) to retrieve the field value for package custom data.

Returns string

Input Parameters key {string} — Key value for the field specified as the parameter input argument with [addPackageParameter\(parameter,sourceField\)](#).

Parent object [Package](#)

Example

```

1 | function calculateRates(input, output)

```

```

2  {
3      ...
4      var shippingDef = input.getShippingDefinition();
5      var allPkgs = shippingDef.getPackages();
6
7      for (var i = 0; i < allPkgs.length; i++)
8      {
9          var thisPkg = allPkgs[i];
10         var thisPkgDesc = thisPkg.getCustomData('pkg_description');
11         ...
12     }
13     ...
14 }

```

getCustomData(key,def)

Function Declaration string getCustomData(string key, string def)

Type Object method

Description Returns the custom data for an package key value or returns a default value. Use this method to retrieve a value for a package field added with [addPackageParameter\(parameter,sourceField\)](#) or, if the key does not exist, use a default value.

You can use this method or use [getCustomData\(key\)](#) to retrieve the field value for package custom data.

Returns string

Input Parameters

- key {string} — Key for the item field specified as the parameter input argument with [addPackageParameter\(parameter,sourceField\)](#).
- def {string} — Field value to return if the key does not exist.

Parent object [Package](#)

Example

```

1  function calculateRates(input, output)
2  {
3      ...
4      var shippingDef = input.getShippingDefinition();
5      var allPkgs = shippingDef.getPackages();
6
7      var defPkgDesc = "String to describe all packages if a description is not present.";
8
9      for (var i = 0; i < allPkgs.length; i++)
10     {
11         var thisPkg = allPkgs[i];
12         var thisPkgDesc = thisPkg.getCustomData('pkg_description',def);
13         ...
14     }
15     ...
16 }

```

Measure

Type Object

Description The Measure object stores the unit of measure and value for weight and size measurements for the [Item](#) and [Package](#) objects. Use the Measure methods to retrieve the weight and size of items and packages for use with shipping rate calculation.

Parent Object(s) [Item](#), [Package](#)

Child Object(s) None.

The following table lists the Measure object methods:

Method	Return Type	Description
getUnit()	string	Returns unit of measure. The unit of measure depends on the item or package properties. The unit of measure can be one of the following values: - Weight: oz, lb, kg, g - Size Return value is null if the unit is not set.
getValue()	string	Returns the currency value. Can be the individual unit price of an item or the total value of a package. Returns an empty string if the value is not set.

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     if (allPackages[i].getTotalValue().getUnit() == 'euro')
5     {
6         var valueInUSD = convertToUSD(allPackages[i].getTotalWeight().getValue());
7         ...
8     }
9     ...
10 }
```

Amount

Type Object

Description The Amount object stores the currency type and value for the [Item](#) and [Package](#) objects. Use the Amount methods to retrieve item or package currency values to use in shipping rate calculation.



Note: The [Rate](#) object also contains an [Amount](#) object with methods you can use to set the currency and value.

Parent Object(s) [Item](#), [Package](#)

Child Object(s) None.

The following table lists the Amount object methods:

Method	Return Type	Description
getCurrency()	string	Returns the ISO code for the currency. Go to Lists > Accounting > Currencies to view the available ISO codes.
getValue()	string	Returns the currency value. Can be the individual unit price of an item or the total value of a package.

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     if (allPackages[i].getLength().getUnit() == 'in')
5     {
6         var lengthInCM = convertToCentimeters(allPackages[i].getLength().getValue());
7         ...
8     }
9     ...
10 }
```

AddressandContact

Type	Object
Description	Container object that you can use to access the Address and Contact for the following object types: ■ Destination, origin, and return addresses for ShippingRoute ■ Manufacturer for an Item
Parent Object(s)	Item , ShippingRoute
Child Object(s)	Address , Contact

The following table lists the AddressandContact object methods:

Method	Return Type	Description
getAddress()	Address	Returns an Address object that contains the mailing address and company information for an addressee. Returns null if the address is not set.
getContact()	Contact	Returns a Contact object for the associated address. Returns null if the contact is not set.

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     var country = shippingSetups[i].getShippingRoute().getDestinationAddressAndContact().getAddress().getCountry();
5     ...
6 }
```

Address

Type	Object
Description	Container object that contains methods to access destination, origin, and return address properties for a ShippingRoute and the address properties for the manufacturer of an Item. You access this object through AddressandContact. All methods except isResidential() return null if the value is not set.
Parent Object(s)	AddressandContact

Child Object(s) n/a

The following table lists the methods for the Address object:

Method	Return Type
getAddressee()	string NULL
getAddressLine1()	string NULL
getAddressLine2()	string NULL
getAddressLine3()	string NULL
getCity()	string NULL
getCompany()	string NULL
getCountry()	string NULL
getPostalCode()	string NULL
getState()	string NULL
isResidential()	boolean

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     var country = shippingSetups[i].getShippingRoute().getDestinationAddressAndContact().getAddress().getCountry();
5     ...
6 }
```

Contact

Type Object

Description Container object that contains methods to access destination, origin, and return contact properties for a [ShippingRoute](#) and the contact properties for the manufacturer of an [Item](#). You access this object through [AddressandContact](#).

All methods return null if the value is not set.

Parent Object(s) [AddressandContact](#)

Child Object(s) n/a

The following table lists the Contact methods:

Method	Return Type
getCompany()	string NULL
getDepartment()	string NULL
getEmail()	string NULL
getFax()	string NULL

Method	Return Type
getFirstName()	string NULL
getFullName()	string NULL
getLastName()	string NULL
getPager()	string NULL
getPhone()	string NULL

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     var company = shippingSetups[i].getShippingRoute().getDestinationAddressAndContact().getContact().getCompany();
5     ...
6 }

```

ShippingSetup

Type	Object
Description	The ShippingSetup object contains the shipping method data for a sales order or item fulfillment transaction. Use this object to access the unique ID of the shipping setup, the date the items and packages shipped, and the ShippingRoute object for the shipment.
Methods	■ getID() ■ getShipDate() ■ getShippingRoute()
Parent Object(s)	ShippingDefinition
Child Object(s)	ShippingRoute

getID()

Function Declaration	java.lang.Integer getId()
Type	Object method
Description	Returns the unique ID integer value of the shipping setup. NetSuite use this ID to uniquely identify the specific shipping setup. You can use this value, for example, to create rates specifically for the associated shipping setup. This method returns null if the property is not set.
Returns	java.lang.Integer
Input Parameters	None.
Parent object	ShippingSetup

Example

```

1 function calculateRates(input, output)
2 {
3     ...

```

```

4   var shipDate = shippingSetups[i].getShipDate();
5   var rate = output.getObjectFactory().newRate(shippingSetups[i].getId());
6   ...
7 }

```

getShipDate()

Function Declaration	string getShipDate()
Type	Object method
Description	Returns the date that the items and packages in the shipment shipped. This method returns null if the property is not set.
Returns	string
Input Parameters	None.
Parent object	ShippingSetup

Example

```

1 function calculateRates(input, output)
2 {
3   ...
4   var shipDate = shippingSetups[i].getShipDate();
5   var rate = output.getObjectFactory().newRate(shippingSetups[i].getId());
6   ...
7 }

```

getShippingRoute()

Function Declaration	ShippingRoute getShippingRoute()
Type	Object method
Description	Returns a ShippingRoute object. This method returns null if there is no shipping information for the associated sales order or item fulfillment transaction.
Returns	ShippingRoute
Input Parameters	None.
Parent object	ShippingSetup

Example

```

1 function calculateRates(input, output)
2 {
3   ...
4   var shippingRoute = shippingSetups[i].getShippingRoute();
5   var country = shippingSetups[i].getShippingRoute().getDestinationAddressAndContact().getContact().getCountry();
6   getRatesByCountry(country, shippingRoute);
7   ...
8 }

```

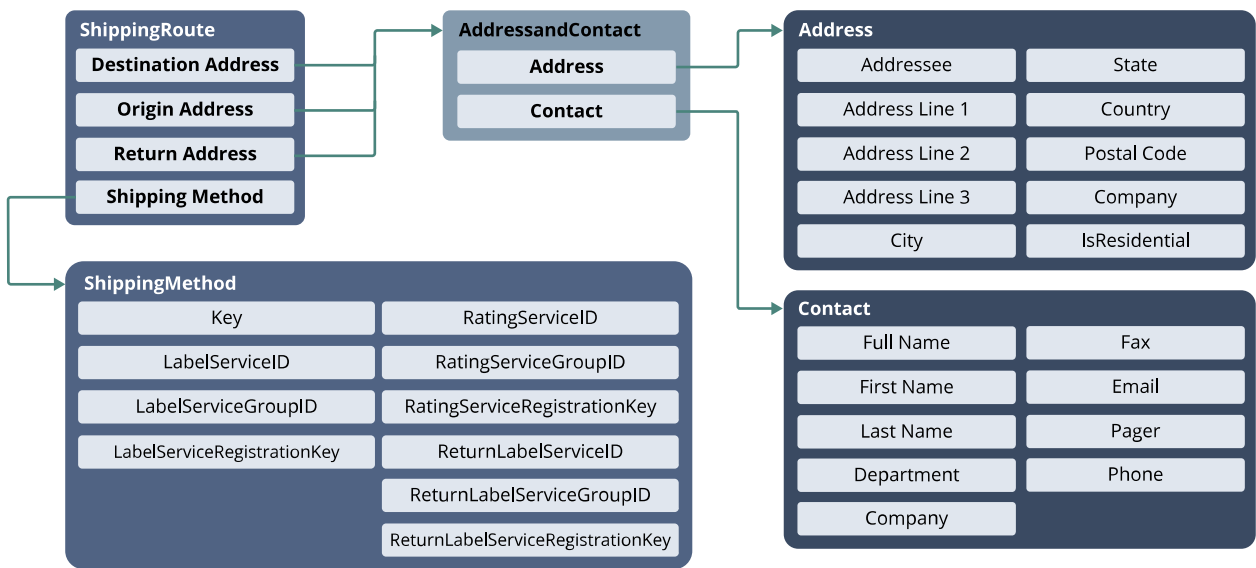
ShippingRoute

Type	Object
-------------	--------

Description	The ShippingRoute object contains the address details and the ShippingMethod object for a sales order or item fulfillment.
Methods	■ getDestinationAddressAndContact() ■ getOriginAddressAndContact() ■ getReturnAddressAndContact() ■ getShippingMethod()
Parent Object(s)	AddressandContact
Child Object(s)	ShippingMethod

ShippingRoute Object Model

The following figure shows the ShippingRoute object model:



getDestinationAddressAndContact()

Function Declaration	<code>AddressandContact getDestinationAddressAndContact()</code>
Type	Object method
Description	Returns an AddressandContact object for the destination address located on the Shipping subtab for a sales order or item fulfillment. This method returns null if there is no destination address specified.
Returns	AddressandContact
Input Parameters	None.
Parent object	ShippingRoute
Example	

```
1 | function calculateRates(input, output)
```

```

2 | {
3 |     ...
4 |     var destinationAddr = shippingSetups[i].getShippingRoute().getDestinationAddressAndContact();
5 |     var sourceAddr = shippingSetups[i].getShippingRoute().getOriginAddressAndContact();
6 |     var weight = shippingSetups[i].getTotalWeight();
7 |
8 |     var rate = getRatesByDistance(sourceAddr,destinationAddr,weight);
9 |     ...
10| }

```

getOriginAddressAndContact()

Function Declaration	AddressAndContact getOriginAddressAndContact()
Type	Object method
Description	Returns an AddressandContact object for the originating shipping address located on the Shipping Address subtab of the Company Information page. This method returns null if there is no destination address specified.
Returns	AddressandContact
Input Parameters	None.
Parent object	ShippingRoute

Example

```

1 | function calculateRates(input, output)
2 | {
3 |     ...
4 |     var destinationAddr = shippingSetups[i].getShippingRoute().getDestinationAddressAndContact();
5 |     var sourceAddr = shippingSetups[i].getShippingRoute().getOriginAddressAndContact();
6 |     var weight = shippingSetups[i].getTotalWeight();
7 |
8 |     var rate = getRatesByDistance(sourceAddr,destinationAddr,weight);
9 |     ...
10| }

```

getReturnAddressAndContact()

Function Declaration	
Type	Object method
Description	Returns an AddressandContact object for the return shipping address located on the Return Address subtab of the Company Information page. This method returns null if there is no destination address specified.
Returns	AddressandContact
Input Parameters	None.
Parent object	ShippingRoute

Example

```

1 | function calculateRates(input, output)
2 | {

```

```

3      ...
4      var destinationAddr = shippingSetups[i].getShippingRoute().getDestinationAddressAndContact();
5      var returnAddr = shippingSetups[i].getShippingRoute().getOriginAddressAndContact();
6      var weight = shippingSetups[i].getTotalWeight();
7
8      var rate = getReturnRatesByDistance(destinationAddr,returnAddr,weight);
9      ...
10 }

```

getShippingMethod()

Function Declaration	ShippingMethod getShippingMethod()
Type	Object method
Description	Returns the ShippingMethod object associated with the shipping setup. Use this method to access the methods available to the ShippingMethod object.
Returns	ShippingMethod
Input Parameters	None.
Parent object	ShippingRoute

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     var serviceId = shippingSetups[i].getShippingRoute().getShippingMethod().getRatingServiceId();
5     ...
6 }

```

ShippingMethod

Type	Object
Description	Contains the properties of the service groups and services that the user selects on the Shipping Item for the transaction. Use these values to define how the plug-in implementation connects to an external shipping partner account to generate rates and shipping labels.

The ShippingMethod object holds properties for the following service types:

- **Rating service.** Rating service values are determined by the service group, service, and external shipping partner registration on the **Shipping Rate** subtab of the Shipping Item page.
- **Label service.** Label service values are determined by the service group, service, and external shipping partner registration under **Shipping Partner Labels** on the **Shipping Label** subtab of the Shipping Item page.
- **Return label service.** Return label service is determined by the service group selected for the Label Service and the service and external partner registration on the **Shipping Label** subtab of the Shipping Item page.



Note: This object can contain services from different service groups, depending on how the Shipping Item is configured.

For more information about registering an external shipping partner account, see [Register Shipping Partner](#).

Parent Object(s) [ShippingRoute](#)

Child Object(s) n/a

The following table describes the ShippingMethod methods:

Method	Return Type	Description
getKey()	string NULL	Returns the internal NetSuite ID for the associated shipping item. Use this value with nlapiLoadRecord(type, id, initializeValues) to load the shipping item record.
getLabelServiceGroupId()	string NULL	ID of the service group selected for shipping label integration on the Shipping Item.
getLabelServiceId()	string NULL	ID of the service selected for shipping label integration on the Shipping Item.
getLabelServiceRegistrationKey()	string NULL	Returns the key of the registration record selected for shipping labels on a shipping item.
getLabelServiceRegistrationCredentialsKey()	string NULL	Use this method to retrieve the key of the custom record used for storing user credentials on the registration record selected for shipping labels on a shipping item. This method allows you to retrieve the key of the credentials custom record without the need to load the whole registration record.
getRatingServiceGroupId()	string NULL	ID of the service group selected for the shipping partner rate on a shipping item.
getRatingServiceId()	string NULL	ID of the service selected for the shipping partner rate on a shipping item.
getRatingServiceRegistrationKey()	string NULL	Returns the key of the registration record selected for the shipping rate on a shipping item.
getRatingServiceRegistrationCredentialsKey()	string Null	Use this method to retrieve the key of the custom record used for storing user credentials on the registration record selected for the shipping rate on a shipping item. This method allows you to retrieve the key of the credentials custom record without the need to load the whole registration record.
getReturnLabelServiceId()	string NULL	ID of the service selected for return label integration on the Shipping Item.
getReturnLabelServiceRegistrationKey()	string NULL	Returns the key of the registration record selected for return labels on a shipping item.
getReturnLabelServiceRegistrationCredentialsKey()	string NULL	Use this method to retrieve the key of the custom record used for storing user credentials on the registration record selected for return labels of a shipping item. This method allows you to retrieve the credentials custom record key without the need to load the whole registration record.

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     var serviceId = shippingSetups[i].getShippingRoute().getShippingMethod().getRatingServiceId();
5     ...

```

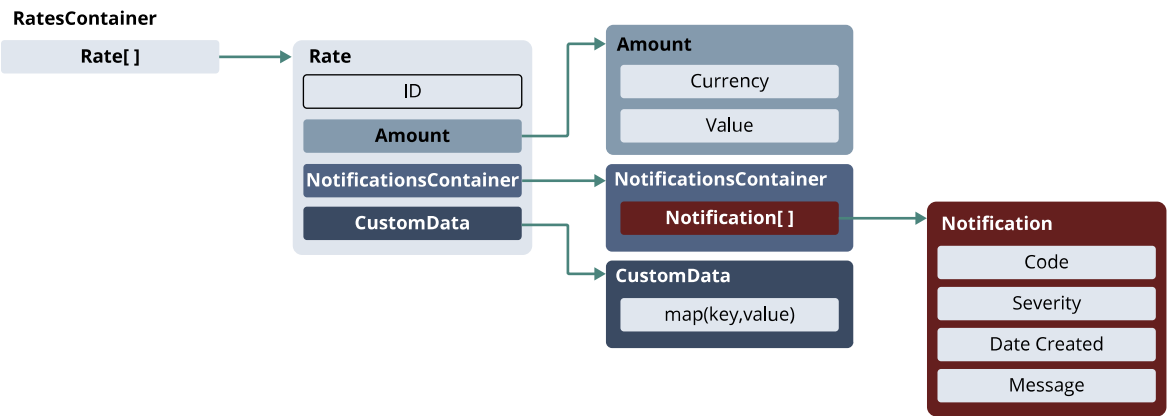
```
6 | }
```

CalculateRatesOutput

Type	Interface output object
Description	The CalculateRatesOutput object is the output object for the calculateRates(CalculateRatesInput input, CalculateRatesOutput output) interface function.
Methods	■ getRatesContainer() ■ getNotificationsContainer() ■ getObjectFactory()
Child Objects	■ RatesContainer , NotificationsContainer

CalculateRatesOutput Object Model

The following figure shows the CalculateRatesOutput object model:



getRatesContainer()

Function Declaration	RatesContainer getRatesContainer()
Type	Object method
Description	Returns the RatesContainer object. Use this method to add shipping rates with addRate(rate) or retrieve shipping rates with getRates() .
Returns	RatesContainer
Input Parameters	None.
Parent object	CalculateRatesOutput

Example

```
1 | function calculateRates(input, output)
2 | {
3 |     ...
```



```

4      // helper function to get the total shipping amount
5      var amount = getShippingTotal();
6
7      var rate = output.getObjectFactory().newRate('letter');
8      rate.setAmount(amount);
9      output.getRatesContainer().addRate(rate);
10     ...
11 }

```

RatesContainer

Type	Object
Description	Container object that includes the methods to add rates to the interface output object or retrieve existing rates.
Methods	■ addRate(rate) ■ getRates()
Parent Object(s)	CalculateRatesOutput
Child Object(s)	Rate

addRate(rate)

Function Declaration	void addRate(Rate rate)
Type	Object method
Description	Add a Rate object to RatesContainer . You must create the rate with newRate(id) and use setAmount(amount) to set the amount of the rate before you can add it.
Returns	void
Input Parameters	rate { Rate } — Rate object that sets the ID and amount for the rate. This method fails if the Rate object is null.
Parent object	RatesContainer

Example

```

1  function calculateRates(input, output)
2  {
3      ...
4      // helper function to get the total shipping amount
5      var amount = getShippingTotal();
6
7      var rate = output.getObjectFactory().newRate(1);
8      rate.setAmount(amount);
9      output.getRatesContainer().addRate(rate);
10     ...
11 }

```

getRates()

Function Declaration	Rate[] getRates()
Type	Object method

Description	Returns the array of Rate objects added to RatesContainer with addRate(rate) . If no rates have been set, the method fails.
Returns	Rate[]
Input Parameters	None.
Parent object	RatesContainer

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     var rates = output.getRatesContainer().getRates();
5     for (var i = 0; i < rates.length; i++)
6     {
7         thisRate = rates[i];
8         // process one rate
9     }
10    ...
11 }

```

Rate

Type	Object
Description	A Rate object contains an ID property and an Amount object that represents a shipping rate for a shipping method. Use newRate(id) to create the Rate object, use setAmount(amount) to set the amount. Then, add the Rate object to RatesContainer and pass the rates back to NetSuite for use on sales order and item fulfillment transactions. The Rate object also contains a NotificationsContainer that you can use to display notifications to a user regarding rate calculations and a CustomData object to pass field values back to NetSuite.



Note: The ID of the Rate object must match the ID of the [ShippingSetup](#) object to which the rate applies.

Methods	■ getAmount() ■ getCustomData() ■ getId() ■ getNotifications() ■ setAmount(amount) ■ setCustomData(customData) ■ setID(id)
Parent Object(s)	RatesContainer
Child Object(s)	Amount , NotificationsContainer

getAmount()

Function Declaration	Amount getAmount()
Type	Object method

Description	Returns the Amount object assigned to a Rate object. The method returns null if no Amount object has been set.
Returns	Amount
Input Parameters	None.
Parent object	Rate

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     var rates = output.getRatesContainer().getRates();
5     var amount = rates[1].getAmount();
6     ...
7 }

```

getCustomData()

Function Declaration	CustomData getCustomData()
Type	Object method
Description	Returns the CustomData object associated with a Rate object. Use this method to access the CustomData object and then set custom data on a Rate object. For more information about setting custom data on a Rate object, see setCustomData(customData).
Returns	CustomData
Input Parameters	None.
Parent object	Rate

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     // set custom item fulfillment field 'custbodyshipa_rm_deliverydate' with current date
5     var deliveryDate = new Date();
6     deliveryDate.setDate(deliveryDate.getDate() + 2);
7     rate.getCustomData().setAttribute('custbodyshipa_rm_deliverydate', deliveryDate.toString());
8     ...
9 }

```

getId()

Function Declaration	java.lang.Integer getId()
Type	Object method
Description	Returns the ID property for a Rate object.
Returns	java.lang.Integer
Input Parameters	None.

Parent object [Rate](#)

Example

```
1 function calculateRates(input, output)
2 {
3     ...
4     var rates = output.getRatesContainer().getRates();
5     var rateId = rates[1].getId();
6     ...
7 }
```

setAmount(amount)

Function Declaration void setAmount(Amount amount)

Type Object method

Description Sets the [Amount](#) object assigned to a [Rate](#) object. You must create the Amount object before you assign it to the Rate object.

Returns void

Input Parameters amount {[Amount](#)} — Amount object to assign to a Rate object.

Parent object [Rate](#)

Example

```
1 function calculateRates(input, output)
2 {
3     ...
4     // helper function to get the total shipping amount
5     var amount = getShippingTotal();
6
7     var rate = output.getObjectFactory().newRate(1);
8     rate.setAmount(amount);
9     output.getRatesContainer().addRate(rate);
10    ...
11 }
```

setCustomData(customData)

Function Declaration void setCustomData(CustomData customData)

Type Object method

Description Associates a [CustomData](#) object with a [Rate](#) object. Use this method to set custom data to be passed back to NetSuite by the plug-in implementation. Use custom data to set field values on an item fulfillment or Web store order.

For the Rate object, you can set custom data on any custom field on the item fulfillment or the **Tracking Number** (trackingnumber or trackingnumberurl) fields. You cannot set values on the Shipment Details or Package Details subrecords.

To use this method, complete the following steps:

1. Identify the field on which you want to set a value.
2. On the associated Rate object, use the [setAttribute\(key,value\)](#) method to set the key and value. The key is the field name and the value is the string value for the field.

After a user calculates rates, the field value is populated.

Returns	void
Input Parameters	customData { CustomData } — CustomData object that contains the field name and field value that you want to pass back to NetSuite.
Parent object	Rate

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     // set custom item fulfillment field 'custbodyshipa_rm_deliverydate' with current date
5     var deliveryDate = new Date();
6     deliveryDate.setDate(deliveryDate.getDate() + 2);
7     rate.getCustomData().setAttribute('custbodyshipa_rm_deliverydate', deliveryDate.toString());
8     ...
9 }

```

setID(id)

Function Declaration	void setID(int id)
Type	Object method
Description	Sets the ID property for a Rate object.



Note: The ID of the Rate object must match the ID of the [ShippingSetup](#) object to which the rate applies.

Returns	void
Input Parameters	id {int} — Integer value for the ID property for a Rate object.
Parent object	Rate

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     var rates = output.getRatesContainer().getRates();
5     currRate = rates[1];
6     currRate.setID(2);
7     ...
8 }

```

Amount

Type	Object
Description	The Amount object stores the currency type and value for a Rate object. Use newAmount(value) to create the object before you add it to a Rate object.
Methods	■ getCurrency() ■ getValue()

- [setCurrency\(currency\)](#)
- [setValue\(value\)](#)

Parent Object(s) [Rate](#)

Child Object(s) n/a

getCurrency()

Function Declaration	string getCurrency()
Type	Object method
Description	Returns the currency property of an Amount object.
Returns	string
Input Parameters	None.
Parent object	Amount

Example

```

1 | function calculateRates(input, output)
2 | {
3 |     ...
4 | }
```

getValue()

Function Declaration	java.lang.Number getValue()
Type	Object method
Description	Returns the value property value of an Amount object.
Returns	string
Input Parameters	None.
Parent object	Amount

Example

```

1 | function calculateRates(input, output)
2 | {
3 |     ...
4 | }
```

setCurrency(currency)

Function Declaration	void setCurrency(string currency)
Type	Object method
Description	Sets the currency property for an Amount object. The currency property for an Amount object defines the currency type for a shipping rate. Typically, this property is a three character string, for example, 'USD'.

Returns	void
Input Parameters	currency {string} — Currency type for a shipping rate.
Parent object	Amount

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     var newRate = getRatesByCountry(input,output,country);
5     amount = output.getObjectFactory().newAmount(newRate);
6     amount.setCurrency('USD');
7     ...
8 }

```

setValue(value)**Function Declaration**

Type	Object method
Description	Sets the value property for an Amount object. The value property for an Amount object corresponds to the value for a shipping rate. Use this method if you already created an Amount object but want to change the value property.
Returns	void
Input Parameters	value {string} — Value property for an Amount object that corresponds to the value for a shipping rate. Cannot be null.
Parent object	Amount

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4 }

```

processShipment(ProcessShipmentInput input, ProcessShipmentOutput output)

Function Declaration	processShipment(ProcessShipmentInput input, ProcessShipmentOutput output)
Type	Interface function
Description	Define the plug-in implementation functionality user saves an item fulfillment. The functionality defined by this function can include partner-specific functionality on the shipping partner account and generation of shipping documents. The input object ProcessShipmentInput provides the sales order, item fulfillment, and Web store order properties, including all items, packages, address and contact information, and

shipping method properties. The input object also provides the internal key for the transaction that is generated by NetSuite.

This function also defines any shipping documents required by the transaction, and passes them back to NetSuite with the [ProcessShipmentOutput](#) object.

Returns	void
Parameters	■ ProcessShipmentInput ■ ProcessShipmentOutput

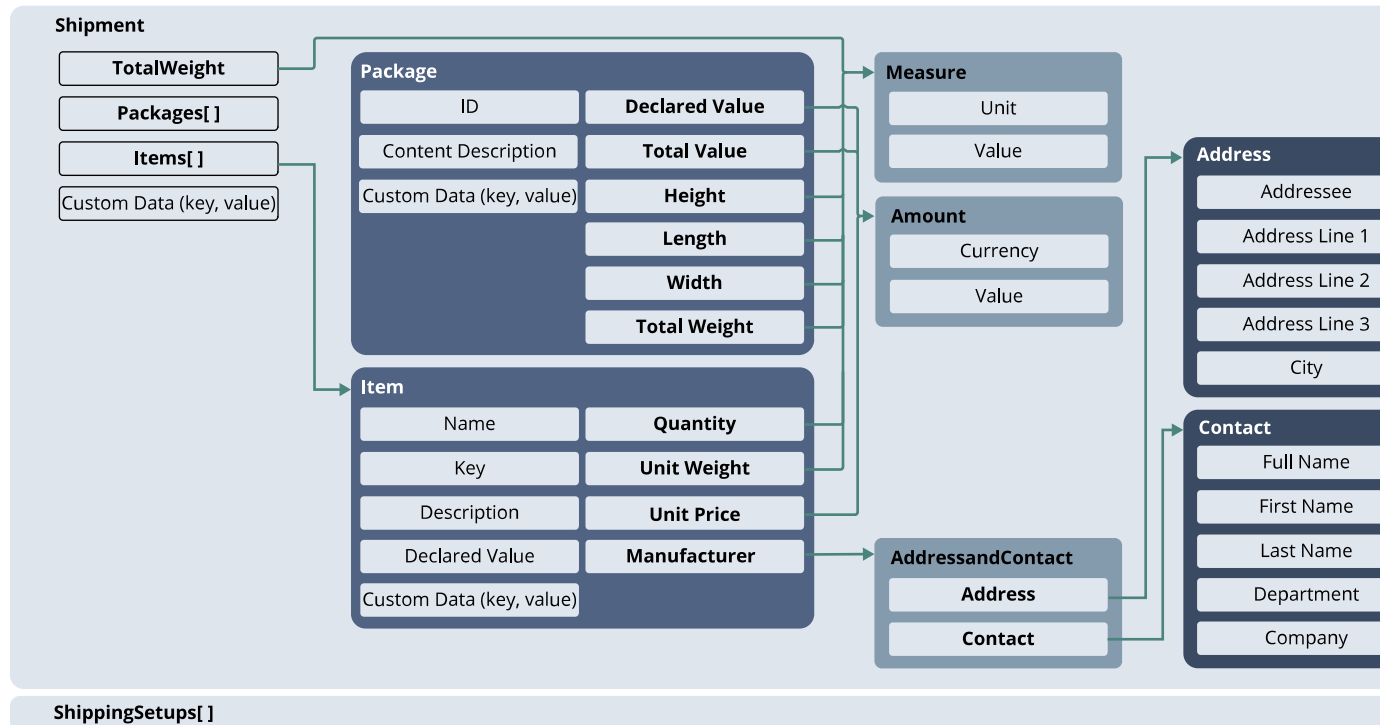
ProcessShipmentInput

Type	Interface input object
Description	Input object for the processShipment(ProcessShipmentInput input, ProcessShipmentOutput output) interface function.
Methods	■ getShippingDefinition() ■ getTransactionKey() ■ getLocale()
Child Objects	■ ShippingDefinition

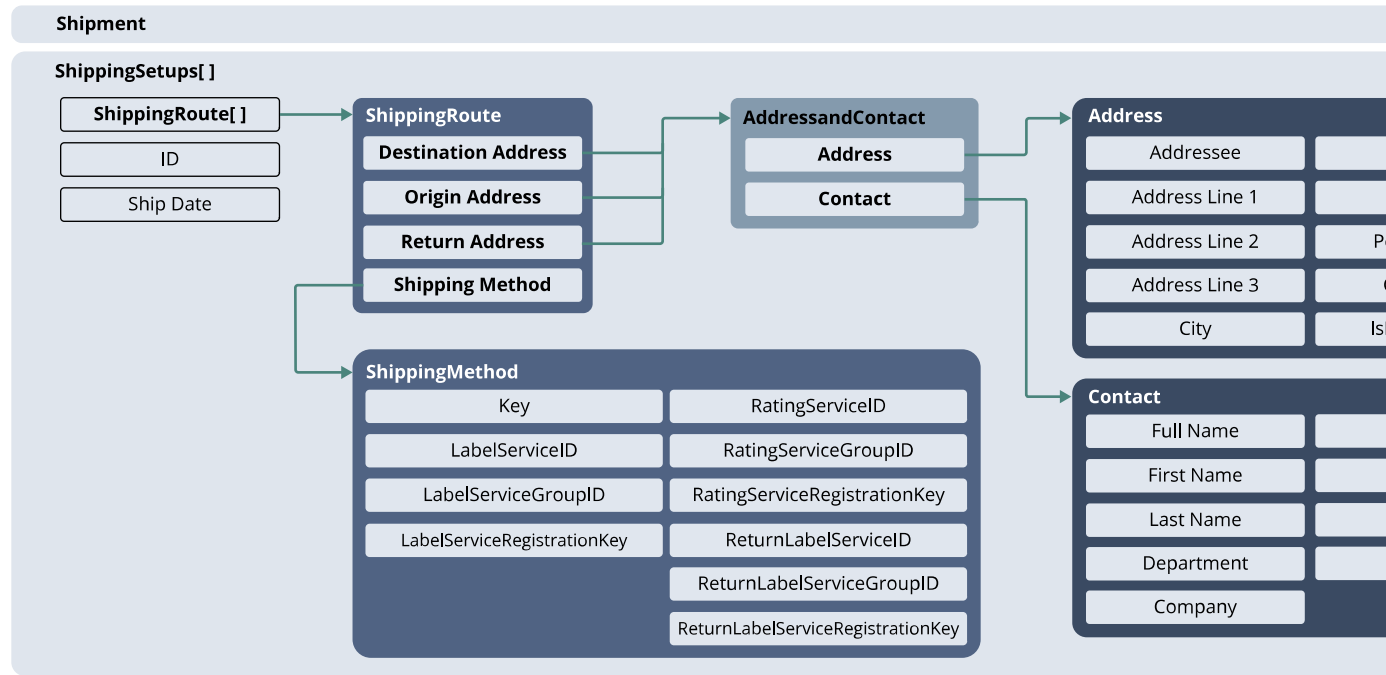
ProcessShipmentInput Object Model

The following figures show the ProcessShipmentInput object model:

ShippingDefinition



ShippingDefinition



getTransactionKey()

Function Declaration

Type	Object method
Description	Returns the unique NetSuite transaction key for an specific item fulfillment transaction. Use this key to track the shipment in the external shipping partner account.
Returns	java.lang.Integer
Input Parameters	None.
Parent object	ProcessShipmentInput

Example

```
1 function processShipment(input, output)
2 {
3     ...
4     var key = input.getTransactionKey();
5     ...
6 }
```

ProcessShipmentOutput

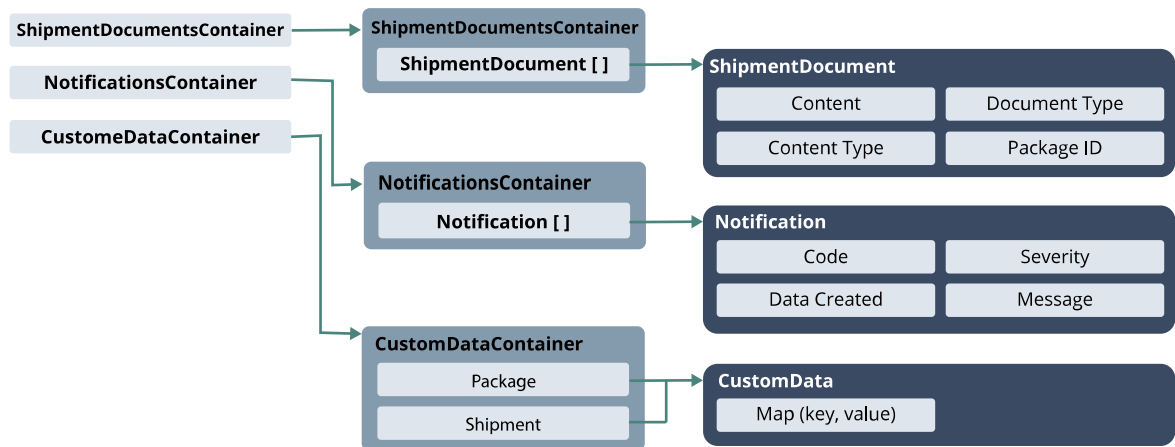
Type	Interface output object
Description	The ProcessShipmentOutput object is the output object for the processShipment(ProcessShipmentInput input, ProcessShipmentOutput output) interface function.

- Methods**
- [getCustomDataContainer\(\)](#)
 - [getShipmentDocuments\(\)](#)
 - [getNotificationsContainer\(\)](#)
 - [getObjectFactory\(\)](#)
- Child Objects**
- [ShipmentDocumentsContainer](#)
 - [NotificationsContainer](#)
 - [CustomDataContainer](#)

ProcessShipmentOutput Object Model

The following figure shows the ProcessShipmentOutput object model:

ProcessShipmentOutput



getCustomDataContainer()

Function Declaration `CustomDataContainer getCustomDataContainer()`

Type Object method

Description Returns a [CustomDataContainer](#) object that contains all custom data for the shipment and all packages passed to the Shipping Partner plug-in implementation. Use this method in the [processShipment\(ProcessShipmentInput input, ProcessShipmentOutput output\)](#) interface function to retrieve all custom data or set custom data for the shipment or a package.

After you use this method, use one of the methods available to the CustomDataContainer object to retrieve custom data for a shipment or packages in an item fulfillment or Web store order.

Returns [CustomDataContainer](#)

Input Parameters None.

Parent object [ProcessShipmentOutput](#)

Example

```

1 | function processShipment(input, output)
2 | {
3 |     var shippingDef = input.getShippingDefinition();

```

```

4      var allPkgs = shippingDef.getPackages();
5      ...
6      var pkgCustomData = output.getCustomDataContainer();
7      for (var i in allPkgs)
8      {
9          var pkg = allPkgs[i];
10         var custData = output.getObjectFactory().newCustomData();
11
12         custData.setAttribute('custrecordship_pkgdata_output', 'Some package data');
13         pkgCustomData.setPackageCustomData(pkg.getId(), custData);
14     }
15     ...
16 }

```

getShipmentDocumentsContainer()

Function Declaration	ShipmentDocumentsContainer getShipmentDocumentsContainer()
Type	Object method
Description	Returns the ShipmentDocumentsContainer container object.
Returns	ShipmentDocumentsContainer
Input Parameters	None.
Parent object	ProcessShipmentOutput

Example

```

1  function processShipment(input, output)
2  {
3      ...
4      var label = output.getObjectFactory().newShipmentDocument('label', 'application/pdf');
5
6      label.setPackageId(1);
7      label.setContent(getPdfLabel());
8      output.getShipmentDocumentsContainer().addShipmentDocument(label);
9      ...
10 }

```

CustomDataContainer

Type	Object
Description	Use this object to set values of some custom fields on the Item Fulfillment transaction. You can set the following types of custom fields: - Transaction custom fields – These are custom fields on the Item Fulfillment transaction body. - Package custom fields – These are custom fields on the Package subrecords. - Shipment custom fields – These are custom fields on the Shipment subrecord.

Use the CustomData object to pass the **key- value** pairs for custom fields you want to set, where **key** is the field name, and value is the **value** to be stored in this field.



Note: You cannot use this method to set values for standard NetSuite fields.

Methods	setTransactionCustomData(CustomData customData) setPackageCustomData(packageID,customData)
----------------	---

- [setShipmentCustomData\(customData\)](#)
- [getTransactionCustomData\(\)](#)
- [getPackageCustomData\(\)](#)
- [getPackageCustomData\(packageID\)](#)
- [getShipmentCustomData\(\)](#)

Parent Object(s) [ProcessShipmentOutput](#)

Child Object(s) [CustomData](#)

setTransactionCustomData(CustomData customData)

Function Declaration void setTransactionCustomData(CustomData customData)

Type Object method

Description Use this method to set the values of custom field on body of the Item Fulfillment transaction.

 **Note:** You cannot use this method to set values for standard NetSuite fields.

Returns void

Input Parameters [CustomData](#) – key-value pairs for custom fields you want to set.

Parent object [CustomDataContainer](#)

Example

```

1  function processShipment(input, output)
2  {
3      ...
4      var transactionCustomData = output.getObjectFactory().newCustomData();
5      transactionCustomData.setAttribute('custbody_field', 'Some value');
6      output.getCustomDataContainer().setTransactionCustomData(transactionCustomData);
7      ...
8  }
```

setPackageCustomData(packageID,customData)

Function Declaration void setPackageCustomData(int packageID, CustomData customData)

Type Object method

Description Associates a new [CustomData](#) object with a [Package](#) object on an item fulfillment or Web store order. This method adds a new key-value custom data pair to the existing map object for the package custom data. Use this method to populate a custom field on the Packages sublist or the Package Details subrecord.

To use this method, complete the following steps:

1. Retrieve the required package ID. Use [getID\(\)](#) on the [Package](#) object to get the ID of the package to which you want to add custom data.
2. Create a new [CustomData](#) object. Use [newCustomData\(\)](#) to create the new object.
3. Set the attributes. Use [setAttribute\(key,value\)](#) to set the attribute key and value. Use the internal NetSuite ID for the custom field on the Packages sublist or on the Package Details subrecord as the key. You can call [setAttribute\(key,value\)](#) multiple times to populate multiple fields for the Package object.

4. Use the ID from step 1 and the object from step 2 in `setPackageCustomData(packageID,customData)`.



Note: You cannot use this method to populate a standard NetSuite field, except for the **Tracking Number** field.

Returns void

Input Parameters packageID {int} – Internal NetSuite ID for a Package object.
[CustomData](#) – key-value pairs for custom fields you want to set.

Parent object [CustomDataContainer](#)

Example

```
1 function processShipment(input, output)
2 {
3     ...
4     var pkgCustomData = output.getCustomDataContainer();
5     var newCustData = output.getObjectFactory().newCustomData();
6     ...
7     newCustData.setAttribute('custrecordshipa_rm_output', 'Some package data');
8     pkgCustomData.setPackageCustomData(pkg.getId(), custData);
9     ...
10 }
```

setShipmentCustomData(customData)

Function Declaration void setShipmentCustomData(CustomData customData)

Type Object method

Description Associates a new [CustomData](#) object with a shipment for an item fulfillment or Web store order. This method adds a new key-value custom data pair to the existing map object for the shipment custom data. Use this method to populate a custom field on the item fulfillment or Web store order or a Shipment Details entry form.

To use this method, complete the following steps:

1. Create a new [CustomData](#) object. Use [newCustomData\(\)](#) to create the new object.
2. Set the attributes. Use [setAttribute\(key,value\)](#) to set the attribute key and value. Use the internal NetSuite IDE for the custom field on the transaction or on Shipment Details subrecord as the key. You can call [setAttribute\(key,value\)](#) multiple times to populate multiple fields for the shipment.



Note: You cannot use this method to populate a standard NetSuite field.

Returns void

Input Parameters [CustomData](#) – key-value pairs for custom fields you want to set.

Parent object [CustomDataContainer](#)

Example

```
1 function processShipment(input, output)
2 {
3     ...
4     var allCustomData = output.getCustomDataContainer();
```

```

5   var shipCustData = output.getObjectFactory().newCustomData();
6
7   // add output of helper function to custrecordshipa_rm_shp_output field
8   shipCustData.setAttribute('custrecordshipa_rm_shp_output',addAdditionalDetails());
9   ...
10  }

```

getTransactionCustomData()

Function Declaration	CustomData getTransactionCustomData()
Type	Object method
Description	Returns a CustomData object that contains a Javascript map. This map represents key-value pairs for custom fields on the Item Fulfillment transaction body.
Returns	CustomData
Input Parameters	None.
Parent object	CustomDataContainer

getPackageCustomData()

Function Declaration	map<int,CustomData> getPackageCustomData()
Type	Object method
Description	Returns a Javascript map that contains key-value pairs for the package custom data. This method returns the map data in the following format: <pre> 1 [packageID1] = CustomData object 2 [packageID2] = CustomData object 3 ... 4 [packageIDN] = CustomData object </pre> The package ID represents the internal NetSuite ID for a Package object. Use <code>getID()</code> for the Package object to retrieve the ID for the package. The CustomData object contains a Javascript map that represents all key-value custom data pairs set for the associated Package objects. NetSuite creates the key-value pairs with the package field values added with <code>addPackageParameter(parameter,sourceField)</code> in the <code>getServiceGroups(GetServiceGroupsInput input, GetServiceGroupsOutput output)</code> interface function. For each CustomData object in the map, you can retrieve and set custom data values with <code>getAttribute(key)</code>, <code>getAttributes()</code>, and <code>setAttribute(key,value)</code>.



Note: You can also set new custom data (that was not passed to the plug-in implementation) for a package custom field with `setPackageCustomData(packageID,customData)`.

Returns	map
Input Parameters	None.
Parent object	CustomDataContainer

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var pkgsCustomData = output.getCustomDataContainer().getPackageCustomData();
5     for (var key in pkgsCustomData)
6     {
7         thisPkgID = key;
8         thisPkgValue = pkgsCustomData[key].getAttribute();
9         ...
10    }
11    ...

```

getPackageCustomData(packageID)

Function Declaration CustomData getPackageCustomData(int packageID)

Type Object method

Description Returns a [CustomData](#) object that contains a Javascript map for all key-value pairs for package custom data. Use [getID\(\)](#) on the [Package](#) object to retrieve the ID for the package. NetSuite creates the key-value pairs with the package field values added with [addPackageParameter\(parameter,sourceField\)](#) in the [getServiceGroups\(input,output\)](#) interface function.

For a CustomData object, you can retrieve and set custom data values with [getAttribute\(key\)](#), [getAttributes\(\)](#), and [setAttribute\(key,value\)](#).



Note: You can also set new custom data (that was not passed to the plug-in implementation) for a package custom field with [setPackageCustomData\(packageID,customData\)](#).

Returns [CustomData](#)

Input Parameters packageID {int} — Internal NetSuite ID for a Package object.

Parent object [CustomDataContainer](#)

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var allCustomData = output.getCustomDataContainer();
5     var shippingDef = input.getShippingDefinition();
6     var allPkgs = shippingDef.getPackages();
7
8     for (var i in allPkgs)
9     {
10        // get CustomData object for this package's custom data
11        var thisPkgCustomDataObj = allCustomData.getPackageCustomData(allPkgs[i].getID());
12        ...
13    }
14    ...
15 }

```

getShipmentCustomData()

Function Declaration CustomData getShipmentCustomData()

Type Object method

Description Returns a [CustomData](#) object that contains a Javascript map for all key-value custom data pairs for a shipment. NetSuite creates the key-value pairs with the shipment field values added with [addShipmentParameter\(parameter,sourceField\)](#) in the [getServiceGroups\(GetServiceGroupsInput input, GetServiceGroupsOutput output\)](#) interface function.

The shipment custom data can be from either the item fulfillment, like the **Memo** field, or from the Shipment Details subrecord.

For each CustomData object in the map, you can retrieve and set custom data values with [getAttribute\(key\)](#), [getAttributes\(\)](#), and [setAttribute\(key,value\)](#).



Note: You can also set new custom data (that was not passed to the plug-in implementation) for a package custom field with [setShipmentCustomData\(customData\)](#).

Returns [CustomData](#)

Input Parameters None.

Parent object [CustomDataContainer](#)

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var allCustomData = output.getCustomDataContainer();
5     var shipMemoField = allCustomData.getShipmentCustomData().getAttribute('txn_memo');
6     ...
7 }

```

CustomData

Type Object

Description Contains the custom data for a [Package](#), [Shipment](#), or [Rate](#) object in a Shipping Partner plug-in implementation. The custom data is stored in a CustomData object as key-value pairs.

The type of key depends on whether NetSuite or the plug-in implementation sends the custom data:

- NetSuite sends the custom data. If NetSuite sends the custom data to the plug-in implementation, the key represents a unique identifier for the field. The value is the field value on the transaction. You can access the CustomData object in the [processShipment\(ProcessShipmentInput input, ProcessShipmentOutput output\)](#) interface function.
- Plug-in implementation sends the custom data. If you set the custom data in the plug-in implementation to pass the value back to NetSuite, the key is the internal NetSuite field ID on the transaction that you want to set. You can set the values for custom fields on the [Rate](#) object in the [calculateRates\(CalculateRatesInput input, CalculateRatesOutput output\)](#) interface function or on the [CustomDataContainer](#) object in the [processShipment\(ProcessShipmentInput input, ProcessShipmentOutput output\)](#) interface function

You can also set the value for the **Tracking Number** field on a package.

The object can contain multiple key-value pairs: one for each field value sent to or set by the plug-in implementation for the object type (Package, Shipment, or Rate).

To create and use a CustomData object to pass field values back to NetSuite, complete the following steps:

1. Use `newCustomData()` to create a new CustomData object.



Note: You do not have to create a new object to set custom data on a Rate object.

2. Use `setAttribute(key,value)` to set the key and value.
3. Use `setPackageCustomData(packageID,customData)` or `setShipmentCustomData(customData)` to set custom data when processing a shipment. Use `setCustomData(customData)` to set custom data on a Rate object.

Methods

- `getAttribute(key)`
- `getAttributes()`
- `setAttribute(key,value)`

Parent Object(s) `CustomDataContainer`, `Rate`

Child Object(s) n/a

getAttribute(key)

Function Declaration `string getAttribute(string key)`

Type Object method

Description Returns the field value for the key of a `CustomData` object.

If the CustomData object was sent to the plug-in implementation by NetSuite, the key is the unique identifier specified in `addItemParameter(parameter,sourceField)`, `addPackageParameter(parameter,sourceField)`, or `addShipmentParameter(parameter,sourceField)`.

If the CustomData object was created by the plug-in implementation to set a field value, the key is the internal NetSuite ID for the custom field or **Tracking Number** field ID on the transaction.



Note: For check box fields, this method returns 'T' if the check box is checked and 'F' if the check box is unchecked.

Returns `string`

Input Parameters `key {string}` — Unique identifier for a field value or the internal NetSuite ID for a field.

Parent object `CustomData`

Example

```
1 function processShipment(input, output)
2 {
3     ...
4     var allCustomData = output.getCustomDataContainer();
5     // get shipment value for 'txn_memo' key
6     var shipMemoField = allCustomData.getShipmentCustomData().getAttribute('txn_memo');
7     ...
8 }
```

getAttributes()

Function Declaration `map<int,CustomData> getAttributes()`

Type	Object method
Description	Returns a JavaScript map that contains all key-value pairs for a custom data in a Package, Shipment, or Rate object. Use this method to get all attributes for a specific CustomData object. If the CustomData object was sent to the plug-in implementation by NetSuite, the key is the unique identifier specified in addItemParameter(parameter,sourceField), addPackageParameter(parameter,sourceField), or addShipmentParameter(parameter,sourceField). If the CustomData object was created by the plug-in implementation to set a field value, the key is the internal NetSuite ID for the custom field or Tracking Number field ID on the transaction.
Returns	map
Input Parameters	None.
Parent object	CustomData

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var allCustomData = output.getCustomDataContainer();
5     var shippingDef = input.getShippingDefinition();
6     var allPkgs = shippingDef.getPackages();
7
8     for (var i in allPkgs)
9     {
10        // get CustomData object for this package's custom data
11        var pkgCustomDataObj = allCustomData.getPackageCustomData(allPkgs[i].getID());
12        var map = pkgCustomDataObj.getAttributes();
13        for (var key in map)
14        {
15            thisValue = map[key];
16            ...
17        }
18    }
19    ...
20 }
```

setAttribute(key,value)

Function Declaration	void setAttribute(key,value)
Type	Object method
Description	Sets the key-value pair for a CustomData object. The key is a internal NetSuite ID for a custom field and the key value is the field value to be passed back to NetSuite. This method returns an error if the key is not a valid field ID. You can only set the value for a custom field in a transaction or the Tracking Number field on an item fulfillment or Web store order. You cannot set the value of standard NetSuite fields with this method. For more information about using this method, see CustomData.
Returns	void
Input Parameters	- key {string} — Internal NetSuite field ID for a custom field or the ID for the Tracking Number field on an item fulfillment or Web store order. - value {string} — String that contains the value for the associated key.

Parent object [CustomData](#)

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var allCustomData = output.getCustomDataContainer();
5     var shipCustData = output.getObjectFactory().newCustomData();
6
7     // add output of helper function to custrecordshipa_rm_shp_output field
8     shipCustData.setAttribute('custrecordshipa_rm_shp_output', addAdditionalDetails());
9     ...
10 }
```

ShipmentDocumentsContainer

Type	Object
Description	Contains all ShipmentDocument objects added with addShipmentDocument(shipmentDocument). Use getShipmentDocumentsContainer() to access this object. The plug-in implementation uses this object to pass all shipment documents back to NetSuite. See ShipmentDocument and for more information about shipment documents.
Methods	■ addShipmentDocument(shipmentDocument) ■ getShipmentDocuments()
Parent Object(s)	ProcessShipmentOutput
Child Object(s)	ShipmentDocumentsContainer

addShipmentDocument(shipmentDocument)

Function Declaration	<code>void addShipmentDocument(ShipmentDocument shipmentDocument)</code>
Type	Object method
Description	Adds a ShipmentDocument object to the ShipmentDocumentsContainer container object.
Returns	void
Input Parameters	shipmentDocument { ShipmentDocument } — ShipmentDocument object to add to the container object. Create with newShipmentDocument(documentType,contentType) .
Parent object	ShipmentDocumentsContainer

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var label = output.getObjectFactory().newShipmentDocument('label', 'application/pdf');
5
6     label.setPackageId(1);
7     label.setContent(getPdfLabel());
8     output.getShipmentDocumentsContainer().addShipmentDocument(label);
9     ...
10 }
```

10 | }

getShipmentDocuments()

Function Declaration	ShipmentDocument[] getShipmentDocuments()
Type	Object method
Description	Returns an array of ShipmentDocument objects added to ShipmentDocumentsContainer with addShipmentDocument(shipmentDocument) .
Returns	ShipmentDocument[]
Input Parameters	None.
Parent object	ShipmentDocumentsContainer

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var allDocuments = output.getShipmentDocumentsContainer().getShipmentDocuments();
5
6     for (var i = 0; i < allDocuments.length; i++)
7     {
8         var currDoc = allDocuments[i];
9         // process one document here
10    }
11    ...
12 }
```

ShipmentDocument

Type	Object
Description	A ShipmentDocument defines a single document to include with an item fulfillment transaction. You can define multiple documents for each item fulfillment. For example, you can include a shipping label documents and return shipping label documents. The documents defined for the transaction in the plug-in appear on the saved item fulfillment under Print > Print Shipping Partner Documents. Use newShipmentDocument(documentType,contentType) to create a shipment document and use setContent(content) to add the content as a string. Then use addShipmentDocument(shipmentDocument) to add the document to ShipmentDocumentsContainer.
Methods	■ getContent() ■ getContentType() ■ getDocumentType() ■ getPackageId() ■ setContent(content) ■ setContentType(contentType) ■ setDocumentType(documentType) ■ setPackageId(packageId)
Parent Object(s)	ShipmentDocumentsContainer
Child Object(s)	n/a

ShipmentDocument Properties

The following table describes the ShipmentDocument properties:

Property	Description	Related Methods
Document type	String value for the type of document. This property is required when you create the shipment document.	setDocumentType(documentType) getDocumentType()
Content type	String value for the content type (MIME type). This property is required when you create the shipment document.	setContentType(contentType) getContentType()
Content	String value for the content. For PDFs, use a string encoded in binary base-64.	setContent(content) getContent()
Package Id	Integer value for the package identifier.	setPackageId(packageId) getPackageId()

getContent()

Function Declaration	string getContent()
Type	Object method
Description	Returns the content property of a ShipmentDocument object. This method returns null if the property is not set.
Returns	string NULL
Input Parameters	None.
Parent object	ShipmentDocument

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var shipmentDocument = output.getObjectFactory.newShipmentDocument('custom', 'application/pdf');
5     shipmentDocument.setPackageId(1);
6     shipmentDocument.setContent(getPdfDocument());
7     output.getShipmentDocumentsContainer().addShipmentDocument(shipmentDocument);
8     ...
9     // make some changes with helper function
10    processDocument(shipmentDocument.getContentType(), shipmentDocument.getDocumentType(),
11                    shipmentDocument.getPackageId(), shipmentDocument.getContent());
12    ...
13 }
```

getContentType()

Function Declaration	string getContentType()
Type	Object method
Description	Returns the content type, or MIME type, of a ShipmentDocument object. For example, for a PDF shipment document, this method returns 'application/pdf'.

Returns	string
Input Parameters	None.
Parent object	ShipmentDocument

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var shipmentDocument = output.getObjectFactory.newShipmentDocument('custom', 'application/pdf');
5     shipmentDocument.setPackageId(1);
6     shipmentDocument.setContent(getPdfDocument());
7     output.getShipmentDocumentsContainer().addShipmentDocument(shipmentDocument);
8     ...
9     // make some changes with helper function
10    processDocument(shipmentDocument.getContentType(), shipmentDocument.getDocumentType(),
11                    shipmentDocument.getPackageId(), shipmentDocument.getContent());
12    ...
13 }
```

getDocumentType()

Function Declaration	string getDocumentType()
Type	Object method
Description	Returns the document type of a ShipmentDocument object.
Returns	string
Input Parameters	None.
Parent object	ShipmentDocument

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var shipmentDocument = output.getObjectFactory.newShipmentDocument('custom', 'application/pdf');
5     shipmentDocument.setPackageId(1);
6     shipmentDocument.setContent(getPdfDocument());
7     output.getShipmentDocumentsContainer().addShipmentDocument(shipmentDocument);
8     ...
9     // make some changes with helper function
10    processDocument(shipmentDocument.getContentType(), shipmentDocument.getDocumentType(),
11                    shipmentDocument.getPackageId(), shipmentDocument.getContent());
12    ...
13 }
```

getPackageId()

Function Declaration	java.lang.Integer getPackageId()
Type	Object method
Description	Returns an the integer value of the package ID of a ShipmentDocument object. This method returns null if the property is not set.
Returns	java.lang.Integer NULL

Input Parameters None.

Parent object [ShipmentDocument](#)

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var shipmentDocument = output.getObjectFactory.newShipmentDocument('custom', 'application/pdf');
5     shipmentDocument.setPackageId(1);
6     shipmentDocument.setContent(getPdfDocument());
7     output.getShipmentDocumentsContainer().addShipmentDocument(shipmentDocument);
8     ...
9     // make some changes with helper function
10    processDocument(shipmentDocument.getDocumentType(), shipmentDocument.getDocumentType(),
11                    shipmentDocument.getPackageId(), shipmentDocument.getContent());
12    ...
13 }
```

setContent(content)

Function Declaration void setContent(string content)

Type Object method

Description Sets the content property of a [ShipmentDocument](#) object. For a PDF, the content must be in a binary, base-64 encoded format.

You can use the properties in the [ShippingSetup](#) object (accessible through the [getShippingDefinition\(\)](#) method) to get details on the transaction and send them to the external shipping partner account to generate the content string.

Returns void

Input Parameters content {string} — String value for the content property of a ShipmentDocument object.

Parent object [ShipmentDocument](#)

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var shipmentDocument = output.getObjectFactory.newShipmentDocument('custom', 'application/pdf');
5     shipmentDocument.setPackageId(1);
6     shipmentDocument.setContent(getPdfDocument());
7     output.getShipmentDocumentsContainer().addShipmentDocument(shipmentDocument);
8     ...
9     // reset the object properties
10    shipmentDocument.setDocumentType('label');
11    shipmentDocument.setContentType('text/plain');
12    shipmentDocument.setContent(getTextDocument());
13    ...
14 }
```

setContentType(contentType)

Function Declaration void setContentType(string contentType)

Type Object method

Description	Sets the content type (MIME type) of a ShipmentDocument object. For example, use 'application/pdf' to create a shipment document in PDF format. Use this property to identify the type of application to use to open the shipping document in NetSuite.
Returns	void
Input Parameters	contentType {string} — Content type property of a ShipmentDocument object.
Parent object	ShipmentDocument

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var shipmentDocument = output.getObjectFactory.newShipmentDocument('custom', 'application/pdf');
5     shipmentDocument.setPackageId(1);
6     shipmentDocument.setContent(getPdfDocument());
7     output.getShipmentDocumentsContainer().addShipmentDocument(shipmentDocument);
8     ...
9     // reset the object properties
10    shipmentDocument.setDocumentType('label');
11    shipmentDocument.setContentType('application/msword');
12    shipmentDocument.setContent(getWordDocument());
13    ...
14 }

```

setDocumentType(documentType)

Function Declaration	void setDocumentType(string documentType)
Type	Object method
Description	Sets the document type property for a ShipmentDocument object. For example, use 'label' for a shipping or return label or use 'custom' for a non-standard document type. Use this property to identify the type of document for the shipment document when you generate the content for the content property.
Returns	void
Input Parameters	documentType {string} — Document type property of a ShipmentDocument.
Parent object	ShipmentDocument

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var shipmentDocument = output.getObjectFactory.newShipmentDocument('custom', 'application/pdf');
5     shipmentDocument.setPackageId(1);
6     shipmentDocument.setContent(getPdfDocument());
7     output.getShipmentDocumentsContainer().addShipmentDocument(shipmentDocument);
8     ...
9     // reset the object properties
10    shipmentDocument.setDocumentType('label');
11    shipmentDocument.setContentType('application/msword');
12    shipmentDocument.setContent(getWordDocument());
13    ...
14 }

```


setPackageId(packageId)

Function Declaration	void setPackageId(java.lang.Integer packageId)
Type	Object method
Description	Sets the integer value of the package ID property of a ShipmentDocument object.
Returns	void
Input Parameters	packageId {java.lang.Integer} — Integer value for the package ID property of a ShipmentDocument.
Parent object	ShipmentDocument

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var shipmentDocument = output.getObjectFactory.newShipmentDocument('custom', 'application/pdf');
5     shipmentDocument.setPackageId(1);
6     shipmentDocument.setContent(getPdfDocument());
7     output.getShipmentDocumentsContainer().addShipmentDocument(shipmentDocument);
8     ...
9     // reset the object properties
10    shipmentDocument.setDocumentType('label');
11    shipmentDocument.setContentType('application/msword');
12    shipmentDocument.setContent(getWordDocument());
13    ...
14 }

```

voidShipment(VoidShipmentInput input, VoidShipmentOutput output)

Function Declaration	void voidShipment(VoidShipmentInput input, VoidShipmentOutput output)
Type	Interface function
Description	Defines the plug-in implementation functionality when a user deletes an item fulfillment. This function provides access the transaction key for the item fulfillment so the shipping partner can process the cancellation. The interface input object VoidShipmentInput provides the NetSuite-generated key for the transaction to define logic to process the cancellation on the shipping partner external account.
Returns	void
Parameters	■ VoidShipmentInput ■ VoidShipmentOutput

VoidShipmentInput

Type	Interface input object
-------------	------------------------

Description	Input object for the voidShipment(VoidShipmentInput input, VoidShipmentOutput output) interface function.
Methods	■ getTransactionKey() ■ getLocale()
Child Objects	n/a

getTransactionKey()

Function Declaration	java.lang.Integer getTransactionKey()
Type	Object method
Description	Returns the NetSuite transaction key for a cancelled sales order as an integer.
Returns	java.lang.Integer
Input Parameters	None.
Parent object	VoidShipmentInput

Example

VoidShipmentOutput

Type	Interface output object
Description	Output object for the voidShipment(VoidShipmentInput input, VoidShipmentOutput output) interface function. Use this object to display notifications to a NetSuite user when the user cancels a transaction.
Methods	■ getNotificationsContainer() ■ getObjectFactory()
Child Objects	■ NotificationsContainer

Input and Output Object Methods

The input and output objects for each interface function have methods that you can access directly from the object. These available methods include the following:

- [getLocale\(\)](#)
- [getObjectFactory\(\)](#)
- [getNotificationsContainer\(\)](#). Output object method that returns messages to NetSuite. You can call this method from the output object of any interface function or a [Rate](#) object.

getLocale()

Function Declaration	void getLocale()
-----------------------------	------------------

Type	Object method
Description	Input object method that returns the browser locale string for the current NetSuite session in the format <locale>_<country code>, for example, 'en_US'. Use this method to customize the plug-in implementation depending on the current browser locale. You can call this method from the input object of any interface function.
Returns	string
Input Parameters	None.
Parent object(s)	GetServiceGroupsInput , DisplayRegistrationInput , ProcessRegistrationInput , CalculateRatesInput , ProcessShipmentInput , VoidShipmentInput

Example

```

1 function getServiceGroups(GetServiceGroupsInput input, GetServiceGroupsOutput output)
2 {
3     ...
4     var locale = input.getLocale();
5     ...
6 }
```

getObjectFactory()

Function Declaration

Type	Object method
Description	Output object method that returns an object factory instance that you use to create new objects in a shipping partner plug-in implementation. The objects you can create depend on the interface function context.
Returns	ObjectFactory
Input Parameters	None.
Parent object	GetServiceGroupsOutput , DisplayRegistrationOutput , ProcessRegistrationOutput , CalculateRatesOutput , ProcessShipmentOutput , VoidShipmentOutput

The following table shows the types of objects that you can create and the related parent objects for the getObjectFactory() method.

Object Type	ObjectFactory Method	Calling Object
Amount	newAmount(value)	CalculateRatesOutput
CustomData	newCustomData()	ProcessShipmentOutput , Rate
CustomDataDescriptor	newCustomDataDescriptor()	GetServiceGroupsOutput
Notification	newNotification()	All interface function output objects for a Shipping Partner plug-in implementation and Rate .
Rate	newRate(id)	CalculateRatesOutput
Service	newService(id)	GetServiceGroupsOutput
ServiceGroup	newServiceGroup(id)	GetServiceGroupsOutput
ShipmentDocument	newShipmentDocument(documentType, contentType)	ProcessShipmentOutput

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     // Object factory for creating new instances
5     var objectFactory = output.getObjectFactory();
6     var notification = objectFactory.newNotification();
7     ...
8     // Direct call
9     var secondNotification = output.getObjectFactory().newNotification();
10    ...
11 }

```

newAmount(value)

Function Declaration Amount newAmount(java.lang.Number value)

Type Object method

Description Returns a new instance of an [Amount](#) object. Use this method, for example, to add a new currency value to a [Rate](#) object.



Note: The Amount object also stores the currency type and value for the [Item](#) and [Package](#) objects.

Returns [Amount](#)

Input Parameters value {java.lang.Number} — Currency value for the new Amount object.

Parent object [getObjectFactory\(\)](#)

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     var rate = output.getObjectFactory().newRate('letter');
5     var amount = output.getObjectFactory().newAmount(6.65);
6
7     amount.setCurrency('USD');
8     rate.setAmount(amount);
9     ...
10
11 }

```

newCustomData()

Function Declaration

Type Object method

Description Returns a new instance of a [CustomData](#) object. Use this method if you want to create a CustomData object to set field values on an item fulfillment or Web store order during shipment processing in the [processShipment\(ProcessShipmentInput input, ProcessShipmentOutput output\)](#) interface function.

For more information, see [CustomData](#) and [Understanding Custom Data in a Shipping Partner Plug-in Implementation](#).

Returns [CustomData](#)

Input Parameters None.

Parent object [getObjectFactory\(\)](#)

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var pkgCustomData = output.getCustomDataContainer();
5     var custData = output.getObjectFactory().newCustomData();
6
7     custData.setAttribute('custrecordshipa_rm_output', 'Some package data');
8     pkgCustomData.setPackageCustomData(pkg.getId(), custData);
9     ...
10 }
```

newCustomDataDescriptor()

Function Declaration

Type Object method

Description Returns a new instance of a [CustomDataDescriptor](#) object. Use this method if you want to pass transaction field values as custom data to a Shipping Partner plug-in implementation.

For more information, see [CustomDataDescriptor](#) and [Understanding Custom Data in a Shipping Partner Plug-in Implementation](#).

Returns [CustomDataDescriptor](#)

Input Parameters None.

Parent object [getObjectFactory\(\)](#)

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     var customDataDescriptor = output.getObjectFactory().newCustomDataDescriptor();
5     customDataDescriptor
6         .addShipmentParameter('txn_memo', 'memo')
7         .addPackageParameter('pkg_description', 'packagedetails.description')
8         .addItemParameter('itm_exclude', 'excludefromraterequest');
9
10    letterGroup.setCustomDataDescriptor(customDataDescriptor);
11    ...
12 }
```

newNotification()

Function Declaration Notification newNotification()

Type Object method

Description	Returns a new instance of a Notification object. Use this method to create a notification to display a message to the NetSuite user during the shipping process for a sales order or item fulfillment.
Returns	Notification
Input Parameters	None.
Parent object	getObjectFactory()

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var notification = output.getObjectFactory.newNotification();
5
6     notification.setCode('0');
7     notification.setSeverity('notice');
8     notification.setMessage('Shipment processed successfully.');
```

newRate(id)

Function Declaration	Rate newRate(java.lang.Integer id)
Type	Object method
Description	Returns a new instance of a Rate object. The Rate object stores the currency value and currency type of a shipping rate. Use addRate(rate) to add the rate to RatesContainer after you create the rate.



Note: The ID of the Rate object must match the ID of the [ShippingSetup](#) object to which the rate applies.

Returns	Rate
Input Parameters	id {string} — Unique string identifier for the Rate object.
Parent object	getObjectFactory()

Example

```

1 function calculateRates(input, output)
2 {
3     ...
4     var rate = output.getObjectFactory().newRate('letter');
5     var amount = output.getObjectFactory().newAmount('6.65');
```

13 | }

newService(id)

Function Declaration	Service newService(string id)
Type	Object method
Description	Returns a new instance of a Service object to associate with a ServiceGroup .
Returns	Service
Input Parameters	id {string} — Unique string identifier for the Service object.
Parent object	getObjectFactory()

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     // create a service group
5     var letterGroup = output.getObjectFactory().newServiceGroup('letter');
6     ...
7     // create the service, set name, and add to service group
8     var firstClass = output.getObjectFactory().newService('firstclass');
9     firstClass.setName('1st Class Mail');
10    lettergroup.addService(firstClass);
11    ...
12 }
```

newServiceGroup(id)

Function Declaration	ServiceGroup newServiceGroup(string id)
Type	Object method
Description	Returns a new instance of a ServiceGroup object.
Returns	ServiceGroup
Input Parameters	id {string} — Unique string identifier for the ServiceGroup object.
Parent object	getObjectFactory()

Example

```

1 function getServiceGroups(input, output)
2 {
3     ...
4     // create a service group
5     var letterGroup = output.getObjectFactory().newServiceGroup('letter');
6     ...
7     // create a service, set name, and add to service group
8     var firstClass = output.getObjectFactory().newService('firstclass');
9     firstClass.setName('1st Class Mail');
10    lettergroup.addService(firstClass);
11    ...
12    // add the service group to the container
13    output.getServiceGroupsContainer().addServiceGroup(letterGroup);
14    ...
15 }
```

15 | }

newShipmentDocument(documentType,contentType)

Function Declaration	ShipmentDocument newShipmentDocument(string documentType, string contentType)
Type	Object method
Description	Returns a new instance of a ShipmentDocument with the specified document type and content type properties.
Returns	ShipmentDocument
Input Parameters	■ documentType {string} — String value for the type of shipment document. For example, use 'label' for a shipping document label or use 'custom' for a non-standard document type. Use this property to identify the type of document for the shipment document when you generate the content for the content property. ■ contentType {string} — Content type (MIME type) of a shipment document. For example, use 'application/pdf' to create a shipment document in PDF format. Use this property to identify the type of application to use to open the shipment document in NetSuite.
Parent object	getObjectFactory()

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var shipmentDocument = output.getObjectFactory.newShipmentDocument('custom', 'application/pdf');
5     shipmentDocument.setPackageId(1);
6     shipmentDocument.setContent(getPdfDocument());
7     output.getShipmentDocumentsContainer().addShipmentDocument(shipmentDocument);
8     ...
9 }
```

getNotificationsContainer()

Function Declaration	NotificationsContainer getNotificationsContainer()
Type	Object method
Description	Returns a NotificationsContainer object. Use this method to add Notification objects to any of the interface output functions or the Rate object with addNotification(notification) or to retrieve the notifications with getNotifications() .
Returns	NotificationsContainer
Input Parameters	None.
Parent object	Rate , GetServiceGroupsOutput , DisplayRegistrationOutput , ProcessRegistrationOutput , CalculateRatesOutput , ProcessShipmentOutput , VoidShipmentOutput

Example

```

1 function processShipment(input, output)
2 {
3     ...
```



```

4      var notification = output.getObjectFactory.newNotification();
5
6      notification.setCode('0');
7      notification.setSeverity('notice');
8      notification.setMessage('Shipment processed successfully.');
```

```

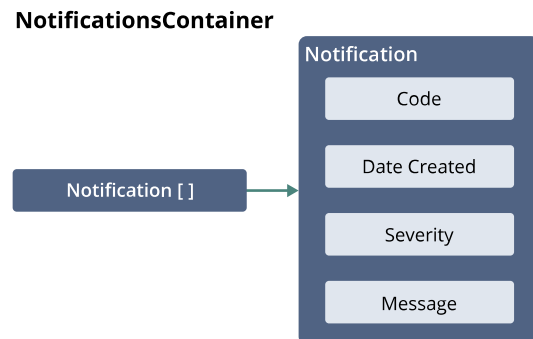
9
10     output.getNotificationsContainer().addNotification(notification);
11     ...
12 }
```

NotificationsContainer

Type	Object
Description	Container object for all Notification objects for the corresponding interface output object or Rate object. Access this object using getNotificationsContainer(). Use this object to add a notification to the plug-in implementation using addNotification(notification) or use this object to access all notifications for an interface function with getNotifications(). This object is available from all interface output functions.
Methods	■ addNotification(notification) ■ getNotifications()
Parent Object(s)	Rate , GetServiceGroupsOutput , DisplayRegistrationOutput , ProcessRegistrationOutput , CalculateRatesOutput , ProcessShipmentOutput , VoidShipmentOutput
Child Object(s)	Notification

NotificationsContainer Object Model

The following figure shows the NotificationsContainer object model:



addNotification(notification)

Function Declaration	void addNotification(Notification notification)
Type	Object method
Description	Add a Notification object to NotificationsContainer. Create the notification with newNotification() before you add it. When you add the notification, NetSuite displays the notification to the NetSuite user.
Returns	void
Input Parameters	notification { Notification } — Notification created with newNotification() cannot be null.

Parent object [NotificationsContainer](#), [NotificationsContainer](#)

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var notification = output.getObjectFactory.newNotification();
5
6     notification.setCode('0');
7     notification.setSeverity('notice');
8     notification.setMessage('Shipment processed successfully.');
```

getNotifications()

Function Declaration `Notification[] getNotifications()`

Type Object method

Description Returns an array of [Notification](#) objects for the notifications that have been added to the interface output object or the [Rate](#) object.

Returns [Notification\[\]](#)

Input Parameters None.

Parent object [NotificationsContainer](#), [NotificationsContainer](#)

Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var notification = output.getObjectFactory.newNotification();
5
6     notification.setCode('0');
7     notification.setSeverity('notice');
8     notification.setMessage('Shipment processed successfully.');
```

Notification

Type Object

Description Displays a message to the NetSuite user during rate calculation or shipment processing on a sales order, item fulfillment, or Web store order. You can create any type of message, including error and informational messages. Create a Notification object with [newNotification\(\)](#) and the notification appears at the top of the NetSuite window when you call [addNotification\(notification\)](#).

The Notification object contains the following properties:

- Code. .
- Severity. NOTICE, WARNING, or ERROR. For most ERROR notifications, the plug-in implementation interrupts the current function and returns to the NetSuite UI to allow the user to fix the error before continuing.
- Message. Message text to display to the user.
- Timestamp. Date and time the Notification object was added with `addNotification(notification)`. The system automatically sets this property.

Methods `getLocale()`

Parent Object(s) `NotificationsContainer`

Child Object(s) n/a

Notification Methods

The following table lists the Notification object methods:

Method	Return Type	Description
<code>getCode()</code>	string NULL	Returns the code property for a notification. Returns null if the code property is not set.
<code>getTimestamp()</code>	java.util.Date	Date and time the Notification object was added with <code>addNotification(notification)</code> .
<code>getMessage()</code>	string NULL	Returns the message text for a notification. Returns null if the code property is not set.
<code>getSeverity()</code>	string NULL	Returns the severity property for a notification. Returns null if the property is not set.
<code>setCode(code)</code>	void	Returns the code property for a notification. Returns null if the property is not set.
<code>setMessage(message)</code>	void	Set the message property for a notification.
<code>setSeverity(severity)</code>	void	Set the severity property for a notification.

Notification Example

```

1 function processShipment(input, output)
2 {
3     ...
4     var notification = output.getObjectFactory.newNotification();
5
6     notification.setCode('0');
7     notification.setSeverity('notice');
8     notification.setMessage('Shipment processed successfully.');
```

SuiteScript Support

After a Shipping Partner plug-in implementation has been installed and set up in your account, you may want to interact with plug-in features using SuiteScript. For information on Shipping Partner records and subrecords that are exposed to SuiteScript, see [Exposed Shipping Partner Records and Subrecords](#).

Exposed Shipping Partner Records and Subrecords

The following record and subrecords are exposed to SuiteScript:

- [Shipping Partner Package](#)
- [Shipping Partner Registration](#)
- [Shipping Partner Shipment](#)

Note that, in addition to scripting that you may want to complete after a plug-in implementation is set up, in some cases the SuiteScript exposures may be useful while developing a plug-in implementation. For example, you may find it useful to script the shipping partner registration record while working on the parts of the implementation that deal with user credentials.

For general details on working with SuiteScript, see [Getting Started with SuiteScript](#). This chapter describes only the Shipping Partner record and subrecords.

 **Note:** The 2014.2 SuiteScript Records Browser does not list the Shipping Partner record or subrecords, because the Shipping Partner plug-in is currently a beta feature.

Shipping Partner Package

The shipping partner package subrecord stores data about a package associated with an item fulfillment record. Typically, this subrecord would include details such as the dimensions of the package and its weight.

The internal ID of this subrecord is *shippingpartnerpackage*.

In the user interface, you access this subrecord from the Packages sublist of an item fulfillment record. For more information, see the help topic [Specify Shipment and Package Details](#).

Note that multiple versions of this form may exist. Typically, forms are created during the development process using standard NetSuite customization tools. Forms are made available to the appropriate service groups by the plug-in implementation.

Supported Script Types

This subrecord is scriptable in both client and server SuiteScript.

The user events are not supported.

Supported Functions

The shipping partner package subrecord is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.



Important: In SuiteScript, subrecords are handled differently from records. Subrecords are represented by the `nlobjSubrecord` object. For general details on working with subrecords, see the help topic [Working with Subrecords in SuiteScript](#).

Field Definitions

The fields available in this subrecord vary depending on the form being used. The following table describes the fields that are available by default.

Internal ID	Type	Label	Required	Notes
customform	select	Custom Form	false	You can use this field to specify the appropriate entry form.
declaredvalue	string	Declared Value	false	
description	string	Description	false	
packageheight	string	Height	false	When searching, refer to this field as <i>height</i> .
packagelength	string	Length	false	When searching, refer to this field as <i>length</i> .
packagewidth	string	Width	false	When searching, refer to this field as <i>width</i> .

Code Sample

The following sample shows a client-side SuiteScript. This sample shows how to add the subrecord to a line in the Packages sublist of an item fulfillment record that is open for editing in the browser.

```

1 | nlapiSelectLineItem("package", 1);
2 | var package = nlapiCreateCurrentLineItemSubrecord("package", "packagedetails");
3 | package.setFieldValue("description", "some description");
4 | package.commit();
5 | nlapiCommitLineItem("package");

```

Shipping Partner Registration

You use the shipping partner registration record as part of the process of registering the partner in NetSuite. The registration process involves gathering account credentials and other details about the shipping partner. After the partner is registered, the account credentials can be used to validate the plug-in implementation with the external account. For many shipping partners, authentication is necessary for tasks such as obtaining real-time shipping rates and generating shipping labels.

The availability of this record varies depending on the plug-in implementation. The record may be optional, required, or not needed.

The internal ID of this record is *shippingpartnerregistration*.

In the user interface, you access this record at Setup > Accounting > Shipping, on the Shipping Registration subtab. For more information on working with this record in the UI, see the help topic [Register a Shipping Partner](#).

Note that multiple versions of this form may exist. Typically, forms are created during the development process. In general, forms are customized using standard NetSuite customization tools. However, the plug-in implementation can add a particular type of field: credential fields. Additionally, the plug-in implementation controls which forms are available to the various service groups.

Supported Script Types

This record is scriptable in both client and server SuiteScript.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

This record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript. Refer to the following table for more details.

Function	Supported?
nlapiCopyRecord	yes
nlapiCreateRecord	yes
nlapiDeleteRecord	yes
nlapiLoadRecord	yes
nlapiSearchRecord	yes
nlapiTransformRecord	no

Field Definitions

The fields available in this record vary depending on the form being used. This section describes the fields available by default, as well as the general handling of credential fields. Note that other custom fields may also exist.

Default Fields

The following table shows the fields available in the record by default.

Internal ID	Type	Label	Required	Notes
addr1	string	Address Line 1	false	
addr2	string	Address Line 2	false	
addr3	string	Address Line 3	false	
city	string	City	false	
company	string	Company	false	
country	string	Country	false	
credentialsrecordkey	string		false	This field is hidden and read only.
customform	select	Custom Form	false	You can use this field to specify the appropriate entry form.
description	string	Description	false	
email	string	Email	false	

Internal ID	Type	Label	Required	Notes
firstname	string	First Name	false	
lastname	string	Last Name	false	
name	string	Name	true	
phone	string	Phone	false	
servicegroup	select	Service Group	true	This field must reference an existing service group that exists for this shipping partner.
shippingpartner	string	Shipping Partner	n/a	
state	string	State	false	
zipcode	string	Zip Code	false	

Credential Fields

In many cases, a plug-in implementation adds credential fields to the shipping partner registration form.

The internal IDs of the credential fields are formatted as follows: `externalfield_name`, where *name* is the actual name of the field, with spaces omitted and capital letters made lowercase. For example, if a field was added called User Password, the SuiteScript internal ID would be `externalfield_userpassword`.

When present, these fields are always required.

Note: The shipping partner registration record works in conjunction with a custom record that is created during the development process. Specifically, both in the UI and in SuiteScript, the credential fields appear to be part of the shipping partner registration form. However, values entered in these fields are stored in the custom record. The registration record stores only the credentials key. Still, for the purposes of scripting to the credential fields, consider them to be part of the shipping partner registration record. For more information about the custom record, see the help topics [Shipping Partner Registration Form](#) and [Step 3. Create Custom Record Type](#).

Usage Notes

When developing a plug-in implementation, you may want to get a value for a credential field using `nlapiGetFieldValue()`. In these cases, the value returned is the hashed value, not the actual value that was entered. The hashed value can be decrypted only inside the `nlapiRequestURLWithCredentials()` call, and only when the following conditions are met:

- The call is invoked by the same plug-in implementation's code, on the same account
- The call targets the domain registered by the plug-in implementation's credential fields.

In other cases, the decrypted value is not available.

The plug-in implementation can never access decrypted credentials because they are sent by NetSuite to the target site inside the web request.

Code Sample

The following code snippets show how to perform common tasks using the shipping partner registration record.

```

1 // change the name of the registration
2 var rec = nlapiLoadRecord("shippingpartnerregistration", <REGISTRATION_INTERNAL_ID>);
3 rec.setFieldValue("name", "new registration name");
4
5 // get credential field
6 rec.getFieldValue("externalfield_<FIELD_NAME>");
7
8 // output: hash of field value
9
10 // update credential value
11 rec.set FieldValue("externalfield_<FIELD_NAME>", "new credential");
12 nlapiSubmitRecord(rec);

```

Shipping Partner Shipment

The shipping partner shipment subrecord stores general details about a shipment associated with an item fulfillment record.

The internal ID of this record is *shippingpartnershipment*.

In the user interface, you access this subrecord from an item fulfillment record. For more information, see the help topics [Specify Shipment and Package Details](#) and [Shipment Details](#).

Note that multiple versions of this form may exist. Typically, forms are created during the development process using standard NetSuite customization tools. Forms are made available to the appropriate service groups by the plug-in implementation.

Supported Script Types

This subrecord is scriptable in both client and server SuiteScript.

The user events are not supported.

Supported Functions

The shipping partner shipment subrecord is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.



Important: In SuiteScript, subrecords are handled differently from records. Subrecords are represented by the `nlobjSubrecord` object. For general details on working with subrecords, see the help topic [Working with Subrecords in SuiteScript](#).

Field Definitions

The fields available in this subrecord vary depending on the form being used.

The following table describes the fields available by default.

Internal ID	Type	Label	Required	Notes
customform	select	Custom Form	false	You can use this field to specify the appropriate entry form.
description	string	Description	false	

Code Sample

The following sample shows a client-side SuiteScript. This sample shows how to add the subrecord to an item fulfillment record that is open for editing in the browser.

```
1 | var shipment = nlapiCreateSubrecord("shipmentdetails");  
2 | shipment.setFieldValue("description", "some description");  
3 | shipment.commit();
```